A Project Report
on
"TI CARE"

Submitted to the
DEPARTMENT OF COMPUTER SCIENCE

In partial fulfilment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

Under the guidance of
Mr. SHERIN MATHEW G

Project done by

ASHIK E A
(Reg No.24222943)

ATHUL KRISHNA P R
(Reg No.24222949)

NIRMALA COLLEGE
MUVATTUPUZHA AUTONOMOUS

(Affiliated to Mahatma Gandhi University)

DEPARTMENT OF COMPUTER SCIENCE

NIRMALA COLLEGE MUVATTUPUZHA
(Autonomous)

OCTOBER 2025

NIRMALA COLLEGE
MUVATTUPUZHA AUTONOMOUS

CERTIFICATE

This is to certify that the project entitled

“TI CARE”

has been successfully carried out by

Ashik E A
(Reg No: 24222943)

Athul Krishna P R
(Reg No: 24222949)

In partial fulfillment of the requirements for the award of the Degree of

MASTER OF COMPUTER APPLICATIONS

of Mahatma Gandhi University, Kottayam during the period 2025-2026

Mr. Sherin Mathew G
INTERNAL GUIDE

Ms. Sherry O. Panicker
HEAD OF THE DEPARTMENT

Place: Muvattupuzha

Date: 10-10-2025

NIRMALA COLLEGE
MUVATTUPUZHA AUTONOMOUS

DECLARATION

We, **ASHIK E A & ATHUL KRISHNA P R**, hereby declare that the project work titled "**TI CARE**" is an authenticated work carried out under the guidance of **Mr. Sherin Mathew G** for the partial fulfillment of the course **MASTER OF COMPUTER APPLICATIONS**. This work has not been submitted for similar purpose anywhere else except to **NIRMALA COLLEGE, MUVATTUPUZHA (AUTONOMOUS).**

We understand that detection of any such copying is liable to be punished in any way the school deems fit.

ASHIK E A

ATHUL KRISNA P R

Place: Muvattupuzha

Date: 10-10-2025

ACKNOWLEDGEMENT

We take this occasion to thank God, Almighty for blessing us with His grace and taking our endeavor to a successful culmination.

We would like to express our special thanks to our **Rev. Dr. JESTIN K KURIAKOSE**, Principal, Nirmala College, Muvattupuzha (Autonomous) for his able guidance and support to completing my project.

We are very grateful **to Ms. SHERRY O. PANICKER**, Head of the department of Computer Science, who is constant source of inspiration and whose advice helped us to complete this project successfully.

We express our profound gratitude to our guide, **Mr. SHERIN MATHEW G**, Associate Professor, Department of Computer Science, for the unfailing and valuable guidance and whole hearted supervision for the conduct of our project.

We also remember with great gratitude for all the encouragement given by our parents, family members, well-wishers and our dear friends in the successful completion of our work.

ASHIK E A

ATHUL KRISHNA P R

CONTENT

Title **Page.No**

1. INTRODUCTION 1
    1.1. INTRODUCTION 2
    1.2. PROBLEM STATEMENT 3
    1.3. SCOPE AND RELEVANCE OF PROJECT 3
    1.4. OBJECTIVES 4

2. SYSTEM ANALYSIS 5
    2.1. INTRODUCTION 6
    2.2. EXISTING SYSTEM 6
    2.3. PROPOSED SYSTEM 7
    2.4. FEASIBILITY STUDY 7
        2.4.1. TECHNICAL FEASIBILITY 8
        2.4.2. ECONOMIC FEASIBILITY 9
        2.4.3. OPERATIONAL FEASIBILITY 10
    2.5. SOFTWARE ENGINEERING PARADIGM APPLIED 10

3. SYSTEM DESIGN 12
    3.1. INTRODUCTION 13
    3.2. DATABASE DESIGN 13
    3.3. OBJECT ORIENTED DESIGN-UML DIAGRAM 19
        3.3.1. ACTIVITY DIAGRAM 19
        3.3.2. SEQUENCE DIAGRAM 22
        3.3.3. USECASE DIAGRAM 25
    3.4. MODULE DESCRIPTION 28
    3.5. INPUT DESIGN 29
    3.6. OUTPUT DESIGN 29

4. SYSTEM ENVIRONMENT 30
    4.1. INTRODUCTION 31
    4.2. SOFTWARE REQUIREMENT SPECIFICATION 31
    4.3. HARDWARE REQUIREMENT SPECIFICATION 32
    4.4. TOOLS, PLATFORMS. 33
        4.4.1. PHP 33
        4.4.2. MySQL 34

5. SYSTEM IMPLEMENTATION 36
5.1. INTRODUCTION 37
5.2. CODING 37
5.2.1. CODING STANDARDS 37
5.3. CODE VALIDATION AND OPTIMIZATION 37
5.3.1. DEBUGGING 37

6. SYSTEM TESTING 38
6.1. INTRODUCTION 39
6.2. TYPES OF TESTING 41
6.2.1. UNIT TESTING 41
6.2.2. SYSTEM TESTING 42
6.2.3. INTEGRATION TESTING 42

7. SYSTEM MAINTENANCE 43
7.1. INTRODUCTION 44
7.2. MAINTENANCE 44

8. FUTURE ENHANCEMENT AND SCOPE FOR FURTHER DEVELOPMENT 45
8.1. INTRODUCTION 46
8.2. MERITS OF THE SYSTEM 46
8.3. LIMITATIONS OF THE SYSTEM 46
8.4. FUTURE ENHANCEMENT OF THE SYSTEM 47

9. CONCLUSION 48
9.1. CONCLUSION 49

10. BIBLIOGRAPHY 50
10.1. Books: 51
10.2. Web Resources: 51

11. ANNEXURES 52
11.1. SCREENSHOTS 53
11.2. CODING 61

ABSTRACT

TiCare is a **web-based application** designed to manage pet health records, with a primary focus on tracking **vaccinations and medication schedules**. Pet owners can register, create detailed profiles for their animals (species, breed, DOB, medical history), and log all necessary health information, including upcoming vaccine due dates and medication times. The system is built using **PHP, HTML/CSS/JavaScript, and MySQL** for data persistence. Its crucial feature is an **automated notification system** implemented with **PHPMailer**, which sends **email alerts** to remind owners about these critical health events.

The application serves to **digitize and centralize** pet health data, which significantly improves record accuracy and supports responsible pet care. By automating reminders, **TiCare** helps **reduce missed vaccinations** and ensures pets adhere to their prescribed medication schedules. The platform's modular design—covering owner registration, pet profile management, and scheduling—makes it a suitable and efficient tool for managing ongoing health needs for both **individual pet owners and veterinary clinics** seeking to streamline their record-keeping processes.

1. INTRODUCTION

1.1. INTRODUCTION

TiCare is a web-based application developed to resolve the systemic issues arising from the manual and fragmented management of pet health records in households and small clinics. This outdated approach frequently results in critical oversights like missed vaccinations and medication errors, significantly compromising a pet's health. **TiCare** addresses this by providing a centralized and secure digital platform where owners can register pets, log detailed medical history, and accurately track essential health schedules. By digitizing records and establishing a single, accessible source of truth, the system dramatically enhances the accuracy and accessibility of crucial health information.

the application's solution is built on a robust web architecture utilizing **PHP** for server-side processing, **MySQL** for persistent data storage, and **HTML/CSS/JavaScript** for a responsive user interface. Its most vital feature is the automated notification mechanism using **PHPMailer**, which sends timely email alerts for upcoming vaccines and medication times. In conclusion, **TiCare's** core goal is to automate and streamline pet health management, thereby reducing human error, enhancing administrative efficiency for both individual owners and veterinary clinics, and ultimately ensuring continuous and responsible pet healthcare.

1.2. PROBLEM STATEMENT

The manual and fragmented nature of pet health record-keeping leads to missed vaccinations and medication errors, posing significant health risks to pets. There is an urgent need for an automated, centralized system like TI CARE to minimize human error and provide proactive alerts for critical care events.

The main problems of the existing system are:

- Fragmented/Manual Records.
- Administrative Overhead.
- Lack of Proactive Alerts.
- Ineffective Treatment
- High Risk of Health Complications

1.3. SCOPE AND RELEVANCE OF PROJECT

The scope of **TiCare** is defined by its core functionality to digitize and centralize pet health management, focusing specifically on user registration, detailed pet profile management, and rigorous tracking of vaccination schedules. This web-based system enhances accountability and reliability by employing the PHP Mailer library to send timely email reminders before vaccination due dates, minimizing the risk of missed immunizations. The application's relevance extends across households, veterinary clinics, and pet care centers, providing a convenient and error-free solution for maintaining systematic health data and supporting responsible, proactive pet healthcare.

1.4. OBJECTIVES

The objectives of this project are:

- To develop a web-based system that stores pet vaccination records in a database.
- To send automated email alerts to pet owners before vaccination due dates.
- To create a responsive, secure, and user-friendly interface accessible through any modern web browser.
- To minimize human error and paperwork in maintaining vaccination records

2. SYSTEM ANALYSIS

2.1. INTRODUCTION

System analysis is a structured method for identifying and solving problems. Analysis implies breaking something into different parts so that the whole can be understood. The definition of system analysis not only process analysis but also that of synthesis, which implies the process of putting parts together to form a new whole. All the activities relating to the life cycle phase must be performed managed and document. To design a system, we need requirements of the system and the specification document is prepared in this phase. The purpose of this document is to specify the functional requirement of the software that is to build. The specifications are intended to guide the activities, relationships and all other objectives.

2.2. EXISTING SYSTEM

The current "system" for managing pet vaccinations is predominantly manual and fragmented. It relies on physical record books provided by veterinarians, individual notes kept by pet owners, and disparate clinic records.

The major drawbacks of this existing system are:

- **Data Loss:** Physical records are susceptible to being lost, damaged, or forgotten.
- **Lack of Reminders:** There is no automated system to remind owners about upcoming or overdue vaccinations.
- **Inaccessibility:** Accessing a pet's complete vaccination history can be difficult, especially if the owner has visited multiple veterinary clinics.
- **Human Error:** Manual data entry and record-keeping are prone to errors and inconsistencies.

2.3. PROPOSED SYSTEM

The proposed **TiCare** system is a web-based application designed to directly overcome the limitations of the manual approach. It will function as a centralized hub for all pet vaccination data, accessible anytime from modern web browsers.

The advantages of the proposed system include:

- **Centralized Database:** All information is stored securely in one place, providing a single source of truth for a pet's health records.
- **Role-Based Access:** The system provides distinct functionalities for Pet Owners (managing their own pets) and Administrators (overseeing the entire system).
- **Automated Reminders:** The system will automatically generate and send reminders for upcoming vaccinations, reducing the chance of missed appointments.
- **Data Integrity and Security:** With a structured database and secure login, the system ensures that data is accurate, consistent, and protected.

2.4. FEASIBILITY STUDY

A feasibility study is made to see if the project on completion will serve the purpose of the organization for the amount of work, time and effort spent on it. Study lets the developer foresee the future of the project and its usefulness.

Finding out whether a new system is required or not. The study is carried out to the best system that meet performance requirements. This entails identification, description and evaluation of candidate system and selection of the best system for the best system for the job.

It simply identifies whether the proposed system is feasible to the organization or not. There are three aspects in the feasibility study portion of the preliminary investigation

- Technical feasibility.
- Economic feasibility.
- Operational feasibility.

2.4.1. TECHNICAL FEASIBILITY

A technical investigation was conducted to evaluate the feasibility of the TiCare system. This stage addresses the following key technical questions:

- Does the necessary technology exist to do what is suggested?

  Yes, the system will be built using standard and mature technologies like Python, SQL, and React, which are widely used for creating reliable and scalable applications.
- Does the proposed equipment have the technical capacity to hold the required data?

  Yes, the data requirements for the system are modest. Modern cloud servers and databases can easily handle the expected data volume and processing load.
- Will the proposed system provide an adequate response to inquiries?

  Yes, as an internal system with a predictable number of users, the architecture will be optimized to ensure fast and reliable performance for all staff members.
- Can the system be upgraded if developed?

  Yes, the system will be designed with a modular architecture, which makes it straightforward to maintain and add new features in the future.

2.4.2. ECONOMIC FEASIBILITY

An economic feasibility study is conducted to provide a thorough cost-benefit analysis of the TiCare project. This evaluation weighs the initial and recurring financial investments against the tangible and intangible returns the system is projected to deliver. The objective is to confirm that the project is not only financially viable but also a strategic investment for the care facility.

Cost Summary:

- Development & Implementation: A one-time cost for building, setting up, and deploying the system.
- Infrastructure: A recurring cost for cloud hosting, servers, and the domain.
- Maintenance: An ongoing cost for updates, security, and technical support.
- Staff Training: A one-time cost to onboard all users onto the new platform.

Benefit Summary:

- Reduced Operational Costs: Lowers spending on paper, printing, and physical storage. Optimizes staff hours, leading to fewer overtime costs.
- Increased Staff Efficiency: Automates manual paperwork and reporting, allowing staff to focus more on resident care.
- Improved Care Quality & Safety: Reduces administrative errors, enhances communication between shifts, and allows for faster response to resident needs.
- Enhanced Compliance & Reporting: Simplifies and speeds up the process of auditing and generating reports, saving significant management time.

2.4.3. OPERATIONAL FEASIBILITY

An operational feasibility study is conducted to determine if the TiCare system will be effectively used by the staff once it is developed and implemented. The study analyzes how the new system fits into the existing workflows of the care facility and assesses potential resistance from users.

The proposed TiCare system is designed to directly support the daily tasks of caregivers and supervisors, replacing inefficient and error-prone manual paperwork. Since the system digitizes and simplifies existing processes rather than forcing a completely new way of working, user adoption is expected to be high. Any initial resistance, which is common with new technology, will be managed through a dedicated training program designed to make staff comfortable and confident with the new tools.

Furthermore, the system is being developed with a strong emphasis on being user-friendly. The interface will be clean, intuitive, and task-oriented, ensuring that even staff with limited computer knowledge can use it effectively with minimal training. The primary operational benefit is providing immediate, real-time access to resident care plans, activity logs, and team communications. This instant access to critical information will improve decision-making and enhance the quality of care. Given these factors, the TiCare system is found to be highly **operationally feasible**.

2.5. SOFTWARE ENGINEERING PARADIGM APPLIED

The project follows the Unified Process (UP) model. The Unified Process is an iterative and incremental software development process framework that is adaptable and well-suited for handling complex projects. Unlike a strict linear model, the UP is use-case driven, architecture-centric, and risk-focused.

The development lifecycle in the Unified Process is organized into four distinct phases, with each phase consisting of one or more iterations:

- **Inception:** The primary goal of this phase is to establish the project's scope and business case. Key activities include identifying stakeholders, gathering initial requirements, and assessing overall feasibility. The aim is to get a general understanding of the system and stakeholder agreement on the project's direction.
- **Elaboration:** In this phase, the core architecture of the system is designed and validated. The requirements are analyzed in greater detail, and significant project risks are identified and mitigated. The outcome is a stable baseline architecture, a more detailed project plan, and a better understanding of the system's requirements.
- **Construction:** This is the phase where the majority of the software is built and tested. The system is developed in a series of iterations, where each iteration adds incremental functionality. The process involves detailed design, coding, unit testing, and integration, resulting in an executable release at the end of each iteration.
- **Transition:** The focus of the final phase is to deliver the completed system to the end-users. This includes activities like beta testing, user training, and data migration.The goal is to ensure that the software is ready for the production environment and meets the expectations of the stakeholders.

Throughout these phases, workflows such as Requirements, Analysis & Design, Implementation, and Testing are active concurrently, though their intensity varies depending on the current phase of the project.

3. SYSTEM DESIGN

3.1. INTRODUCTION

System design is a process of developing specifications for a candidate system that meet the criteria established in the system analysis. Major steps in design are the preparation of the input forms and output reports in a form applicable to user. The main objective of the system design is to use the package easily by any computer operator. System design is the creative act of invention, developing new inputs, a database, offline files, method, procedure and output for processing business to meet an organization objective. System design builds information gathered during the system analysis.

3.2. DATABASE DESIGN

The most important aspect of building an application is the design of tables or the database schema. The data stored in the tables must be organized in some manner, which is meaningful. The overall objective in the process of table design has been to treat data as an organizational, resource and as an integrated whole. The organization of data in a database aim to achieve three major objectives, which are given below.

- Data integration.
- Data abstraction.
- Data independence.

Several degrees of normalization have to be applied during the process of table design. The major aim of the process of normalization is to reduce data redundancy and prevent losing data integrity. Data integrity has to be converted at all levels. Pure normalization can access problem related to storage and retrieval of data.

During the process of normalization, dependence's can be identified which cause serious problems during deletion and updating. Normalizing also hope in simplifying the structure of table

Database design is an important activity in design. The efficiency of the system lies in the efficiency of the database. The database design consists of predetermining number of tables that are to be used and fields that are to be used in each table. The efficiency of the system to retrieve the appropriate data depends on how the fields are specified and the coding mostly depends on format of the database. To group the given data and to give them a logical structure, a database design is necessary. The overall objective in the development of the database is to treat data as an organized resource and is an integrated whole.

In this project we are mainly concentrated into relational database. Relational database stores data in table, which is turn, are composed of rows also known as records, columns also known as fields. The fields in the relational models are.,

PRIMARY KEY-The key which uniquely identifies the records. They also known notify the not null constraint.

FOREIGN KEY-The key references the primary key, is the data inserted in the primary key column of the table.

The general theme is to handle information as an integrated whole with a minimum of redundancy and improved performance. A complete database management system separates definition of data from programs. The concept of data independence is one of the key advantages of a database management system. The choice of a particular database system is complicated. In this project a relational data model is used because of many features it offers. Data directory specifies the major element in the system, and care should be taken

while designing, in order to avoid unnecessary duplication of data. Several tables are maintaining the system to store data those are required for processing of various data as well storing intermediate for the final processed results. Different modules to meet their needs access these stored data.

Designing a database is complete task and the normalization theory is a useful aid in the design process. The process of normalization is concerned with transformation of conceptual schema into computer representation form. There will be need for most databases to grow by adding new attributes and new relations. The data will be used in new ways. Tuples will be added and deleted. Information stored may undergo updating also. New association may also be added. In such situations the performance of a database is entirely depend upon its design. A bad database design may lead to certain undesirable things like:

1. Repetition of information
2. Inability to represent certain information
3. Loss of information

To minimize these anomalies, Normalization may be used. If the database is in a normalized form, the data can be growing without, in most cases, forcing the rewriting application programs. This is important because of the excessive and growing cost of maintaining an organization's application programs and its data from the disrupting effects of database growth. As the quality of application programs increases, the cost of maintaining the without normalization will rise to prohibitive levels. A normalized database can also encompass many related activities of an organization thereby minimizing the need for rewriting the applications of programs. Thus, normalization helps one attain a good database design and there by ensures continued efficiency of database.

Normalization theory is built around the concept of normal forms.A relation is said to be in normal form if it satisfies a certain specified set of constraints. For example, a relation is said to be in first normal form (1NF) if it satisfies the constraint that it contains atomic values only, thus every normalized relation is in 1NF.Numerous normal forms have been defined. Codd defined the first three normal forms.

All normalized relations are in 1NF, some 1NF relations are also in 2NF and some 2NF relations are also in 3NF.2NF relations are more desirable than 1Nf and 3NF are more desirable than 2NF. That is, the database designer should prefer 3NF than 1NF or 2NF.Normalization procedure states that a relation that is in some given normal form can be converted into a set of relations in a more desirable form. The successive reduction of a given collection of relations to some more desirable form. This procedure is reversible. That is, it is always possible to take the output from the procedure and convert them back into input. In this process, no information is lost. So, it is also called “no loss decompositions

First Normal Form

A relation is in first normal form (1NF) if and all its attributes are based on single domain. The objective of normalizing a table is to remove its repeating groups and ensure that all entries of the resulting table have at most single value.

Second Normal Form

A table is said to be second Normal Form (2NF), when it is in 1NF and every attribute in the record is functionally dependent upon the whole key, and not just a part of the key.

TABLES

1. Table Name: admin

Description: To store admin login details

Primary Key: id

Field Name	Data Type	Description
id	Int(11)	Id of Admin
username	varchar(250)	Username
email	varchar(250)	Email for log in
password	varchar(250)	Password for log in

2. Table Name: pet

Description: pet is used for storing pet details.

Primary Key: id

Field Name	Data Type	Description
id	Int(11)	Id
user_id	int (250)	Users id
name	varchar (250)	Name of pet
type	varchar (20)	Type of pet
breed	varchar (20)	Type of breed
date	date	date

3. Table Name: pet_vaccination_records

Description: vaccination_records to store vaccine details.

Primary Key: id

Foreign key: Pet_id

Field Name	Data Type	Description
id	Int (11)	Id
Pet_id	int (11)	Id of the pet
Owner_email	varchar (250)	Email of the owner
Pet_name	varchar (250)	Name of the pet
Vaccine_name	varchar (250)	Name of the vaccine
Reminder_date	datetime	Date of the reminder
duedate	datetime	Date of due
remindersent	Tinyint (1)	Reminder to sent
vaccineimage	Varchar (2500)	Image of vaccine
notes	Varchar (250)	Written notes

4. Table Name: users

Description: users used to store user details.

Primary Key: id

Field Name	Data Type	Description
id	Int (10)	Id of user
username	Varchar (30)	Name of User
email	Varchar (50)	User email
password	Varchar (100)	Password of email
Created_at	timestamp	Current time and date

3.3. OBJECT ORIENTED DESIGN-UML DIAGRAM

3.3.1. ACTIVITY DIAGRAM

Activity diagrams are graphical representation of workflows of stepwise activities and actions with support for choice, iteration and concurrency. UML, activity diagrams are intended to model both computational and organizational process. It shows the overall flow of control.

Symbols in Activity Diagram

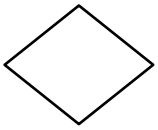
Action State: - It is an automatic state once it started execution. It will come to completion without any interruption at the middle of execution.



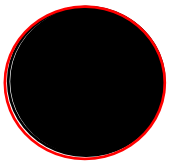
Transition: - When the action or activity state completes the flow of control passes immediately to the next action or activity state. This is represented by using the solid lines.



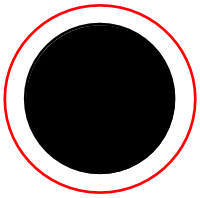
Branching: - It specifies the alternative paths that are taken based on some Boolean expressions.



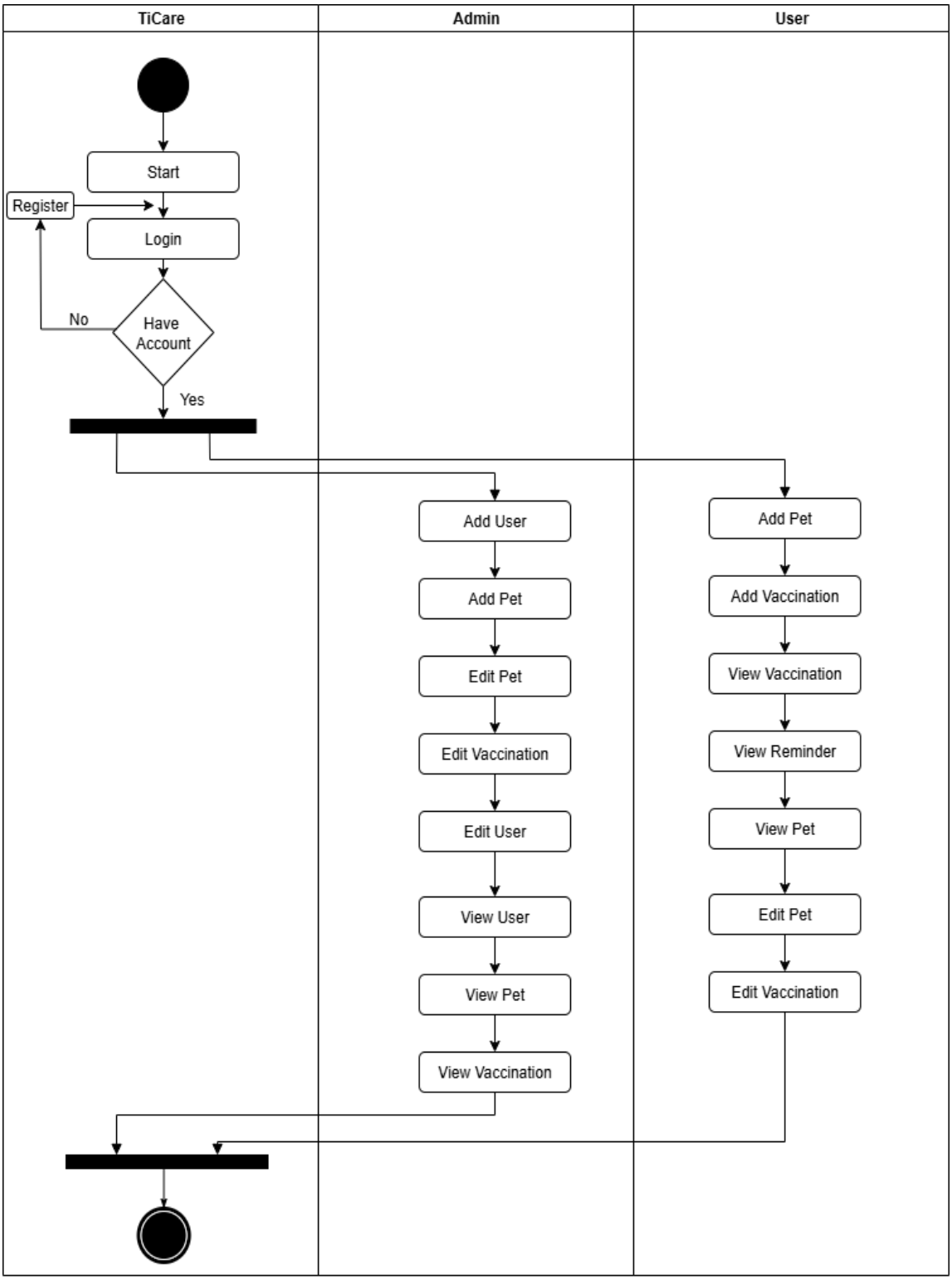
Initial node: - The initial node is a control node from where the activity is invoked. An activity may have more than one initial node.



Final node: - The final node shows the end of the activity. An activity can have more than one final node but the first one reached will stop the activity.



ACTIVITY DIAGRAM

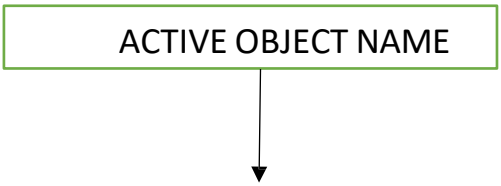


3.3.2. SEQUENCE DIAGRAM

A sequence diagram is an interaction diagram that shows how objects operate with one another and in what order. It is a constant of a message sequence chart. A sequence diagram shows object interactions arranged in time sequence. It depicts the objects and classes involved in the scenario and the sequence of messages exchanged between the objects needed to carry out the functionality of the scenario. Sequence diagrams are typically associated with use case realizations in the logical view of the system under development

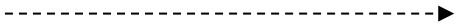
3.3.2.1. Symbols in Sequence Diagram

- **Life lines & life line boxes: -** Active objects can be an object which is an instance of a class and can be drawn with rectangular box called the life line box with its name is specified within the box. The long dashed line tailing the object is called life lines.

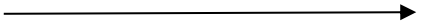


- **Message:** -It is used to illustrate the communication between different active objects. Each message between object is represented with a message, expressions and filled arrow solid lines from the calling active objects life line to the recipient life lines.
- **Synchronous:** - This type of message is used when it is important that a message is received & completed before execution of control flow begins.

- **Return**: -Return message shows that the control flow has return into the calling active objects and the synchronous messages are completed its operations.

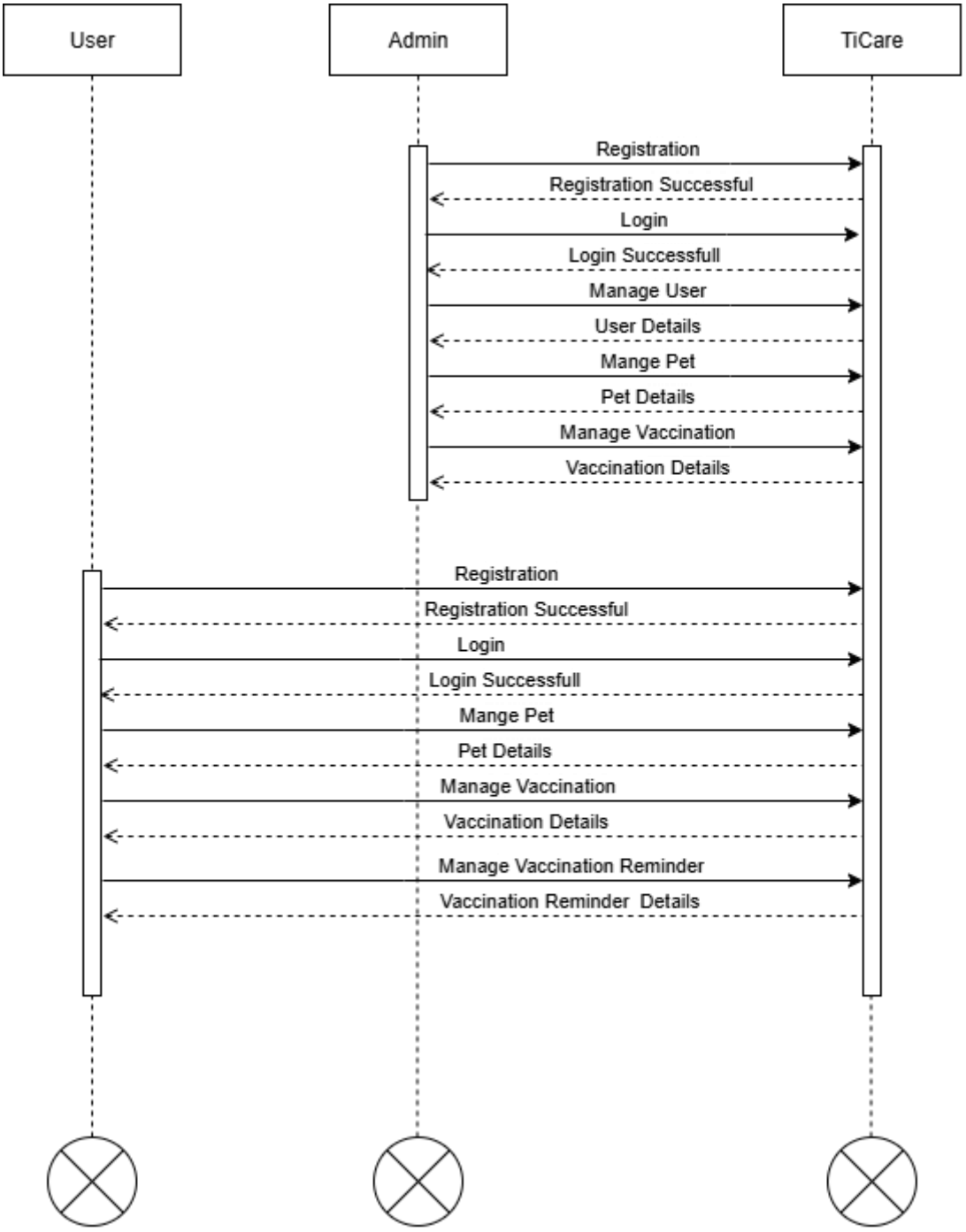


- **Asynchronous**: - This type of message is sent from an active object and waits for a responds. This type of message is used when control flow doesn't lead to interrupted before complete the operation.



- **Flat:** -This type of message is used when there is no distinction between asynchronous and synchronous messages

SEQUENCE DIAGRAM

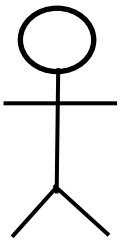


3.3.3. USECASE DIAGRAM

The most important aspect is to capture the dynamic behavior when modelling a system. Dynamic means the behavior of the system when it is operating. Use case diagram is one of the diagrams that describe the dynamic behavior of the system. Use case diagrams are used to gather the requirements of a system including internal and external influences. These requirements are mostly design requirements. Hence, when a system analyzed to gather its functionalities, use cases are prepared and actors are identified. When the initial task is complete, use case diagrams are modelled to present the outside view.

3.3.3.1. Symbols in Use-case Diagram:

An actor represents a coherent set of roles that users of a system play when interacting with the use cases of the system. Actors can be anything-humans, devices, other systems. One physical object can play several roles and so can be modelled by several actors.



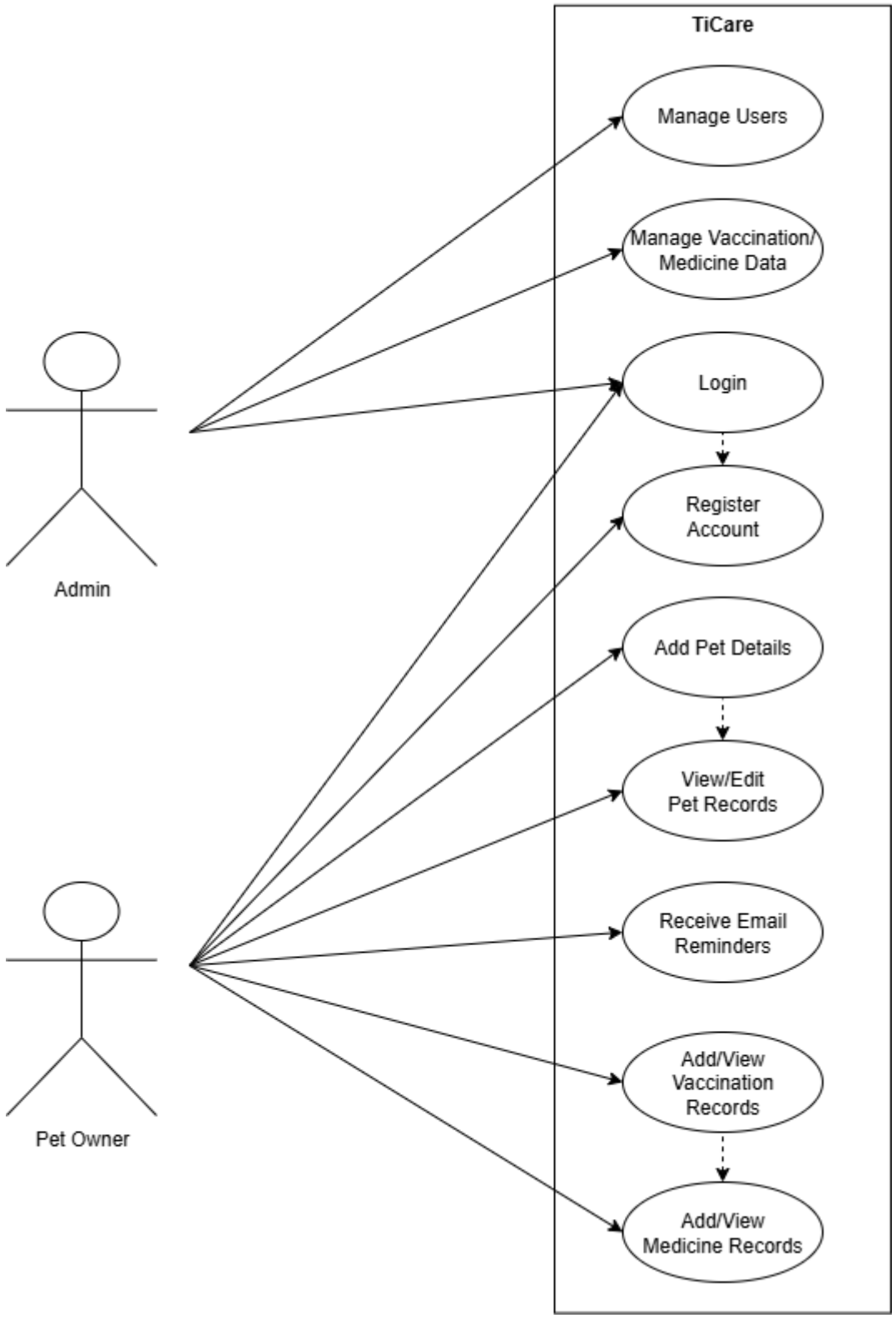
Use cases describe what a system does, not how it does it. A use case contains multiple scenarios, each of which describes a specific flow of events through the use case. Use case behavior is specified by describing the scenarios clearly enough for outsiders to understand.

Connection symbol represents the connection between actor and use cases**.**



Include symbol include the relationship between use cases. One use case(base) includes the functionality of another (inclusion case).

USECASE DIAGRAM



3.4. MODULE DESCRIPTION

The TiCare system is an integrated platform designed to streamline operations, enhance communication, and improve care delivery within a healthcare facility. It is divided into three primary modules based on user roles.

1. Staff / Caregiver Module

- Login & Daily Dashboard: View tasks, schedules, and alerts.
- Access Care Plans: Look up resident instructions and history.
- Log Activities: Record medications, vitals, and notes.
- Team Communication: Send messages and shift handovers.

2. Supervisor / Manager Module

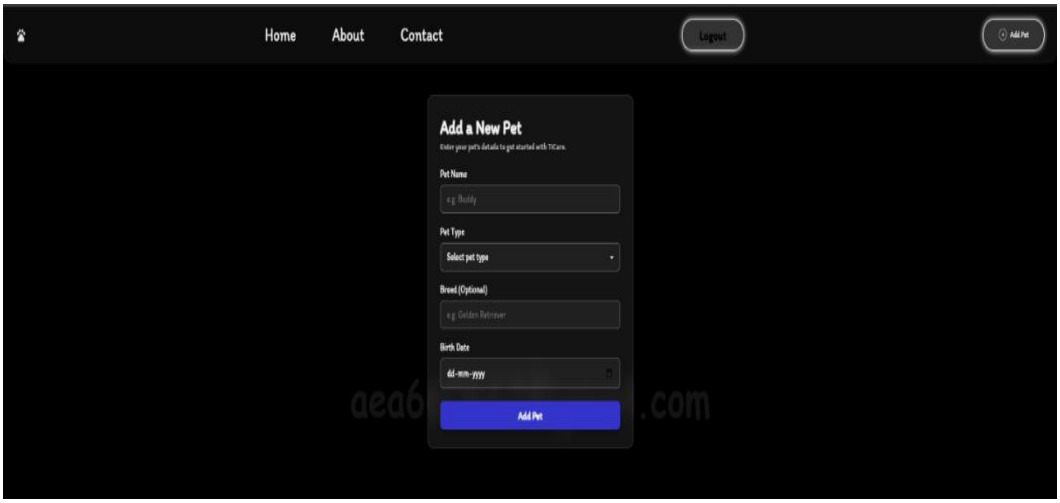
- Team Oversight: Monitor staff activity and resident status.
- Care Plan Management: Create, assign, and update care plans.
- Review Records: Access full resident logs and reports.
- Generate Analytics: Create performance and health reports.

3. Admin Module

- User Management: Add, remove, and manage all staff accounts.
- Facility & Resident Setup: Manage rooms, departments, and resident data.
- System Configuration: Customize workflows and system settings.
- Audit & Security: Ensure data compliance and security.

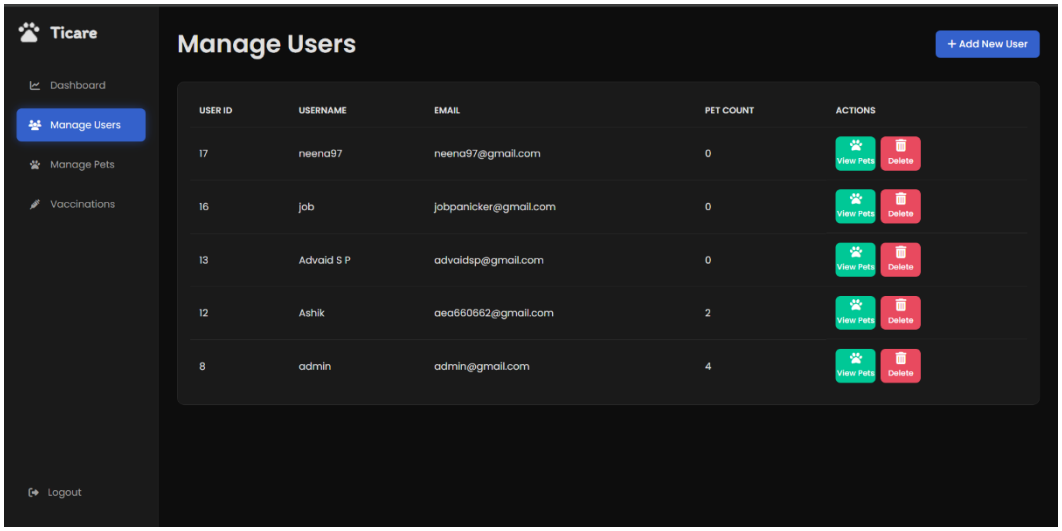
3.5. INPUT DESIGN

Input design is the process of converting user-oriented input to a based format. Inaccurate input data are the most common cause of errors in data processing. Errors entered by data entry operators can be controlled by input design.



3.6. OUTPUT DESIGN

Computer Output is the most important and direct source of information to the user. It is the most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship and helps user decision-making.



4. SYSTEM ENVIRONMENT

4.1. INTRODUCTION

The unique technical and operating characteristics of an IT system and its associated environment, including the hardware, software, firmware, communications capability, organization, and physical location. The system environment is primarily the set of variables that define or control certain aspects of process execution. From the system-management point of view, it is important to ensure the user is set up with the correct values at log in. Most of these variables are set during system initialization.

4.2. SOFTWARE REQUIREMENT SPECIFICATION

After the analyst has collected all the required information regarding the software to be developed, and has removed all completeness, inconsistence, and anomalies from the specification, he starts to systematically organize the requirements in the form of an SRS document. The software developers refer to the SRS document to make sure that they develop exactly what the customer requires. The SRS document helps the maintenance engineers to understand the functionality of the system. An SRS document should be clearly specify.

- Functional requirements
- Non -functional requirements
- Goals of implementation

Software Interface

Operating System: Windows 11

4.3. HARDWARE REQUIREMENT SPECIFICATION

The most common set of requirements defined by any operating system or software application is the physical computer resources, also known as hardware, A hardware requirements list is often accompanied by a hardware compatibility list. This configuration ensures that developers can run local web servers such as XAMPP or WAMP efficiently while coding and testing the PHP and MySQL- based application.

The hardware compatibility lists tested, compatible, and sometimes incompatible hardware devices for a particular operating system or application. Selection of hardware configuration is very important task related to the software development. The processor should be powerful to handle all the operations. The hard disk should have the sufficient capacity to solve the database and the application. The minimum requirement for the implementation of the system is one machine and internet connection. On the client side, users only need a device with basic internet access, such as a desktop, laptop, tablet, or smartphone. Since TI CARE is a web-based system, it can run smoothly on any device with a modern web browser like Google Chrome, Mozilla Firefox, or Microsoft Edge. There is no requirement for additional hardware installation or specialized devices. The simplicity of hardware requirements makes TI CARE highly accessible, cost- effective, and easy to use for both individual pet owners and veterinary clinics.

CPU: Intel(R) Core (TM) i5-13thgen H CPU @ 3.5GHz

RAM: 16GB

Keyboard and Mouse: Standard

4.4. TOOLS, PLATFORMS.

4.4.1. PHP

The PHP Hypertext Pre-processor (PHP) is a programming language that allows web developers to create dynamic content that interacts with databases. PHP is basically used for developing web-based software applications. This tutorial helps you to build your base with PHP. PHP started out as a small opensource project that evolved as more and more people found out how useful it was. Rasmus Lerdorf unleashed the first version of PHP way back in 1994.

- PHP is a recursive acronym for "PHP: Hypertext Pre-processor".
- PHP is a server-side scripting language that is embedded in HTML. It is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.
- It is integrated with a number of popular databases, including MySQL, SQL, Oracle, Sybase, Informix, and Microsoft SQL Server.
- PHP is pleasingly zippy in its execution, especially when compiled as an Apache module on the Unix side. The MySQL server, once started, executes even very complex queries with huge result sets in record-setting time.
- PHP supports a large number of major protocols such as POP3, IMAP, and LDAP. PHP4 added support for Java and distributed object architectures (COM and CORBA), making n-tier development a possibility for the first time.
- PHP is forgiving: PHP language tries to be as forgiving as possible.

Common uses of PHP

- PHP performs system functions, i.e. from files on a system it can create, open, read, write, and close them.
- PHP can handle forms, i.e. gather data from files, save data to a file, through email you can send data, return data to the user.
- You add, delete, modify elements within your database through PHP.
- Access cookies variables and set cookies.
- Using PHP, you can restrict users to access some pages of your website.
- It can encrypt data.

Characteristics of PHP

Five important characteristics make PHP's practical nature possible −

- Simplicity
- Efficiency
- Security
- Flexibility
- Familiarity

4.4.2. MySQL

MySQL, the most popular Open-Source SQL database management system, is developed, distributed, and supported by Oracle Corporation. If that is what you are looking for, you should give it a try. MySQL Server can run comfortably on a desktop or laptop, alongside your other applications, web servers, and so on, requiring little or no attention. If you dedicate an entire machine to MySQL, you can adjust the settings to take advantage of all the memory, CPU power, and I/O capacity available. MySQL can also scale up to clusters of machines, networked together.

MySQL Server was originally developed to handle large databases much faster than existing solutions and has been successfully used in highly demanding production environments for several years. Although under constant development, MySQL Server today offers a rich and useful set of functions.

Its connectivity, speed, and security make MySQL Server highly suited for accessing databases on the Internet.

features of MySQL

- MySQL is released under an open-source license. So you have nothing to pay to use it.
- MySQL is a very powerful program in its own right. It handles a large subset of the functionality of the most expensive and powerful database packages.
- MySQL uses a standard form of the well-known SQL data language.

5. SYSTEM IMPLEMENTATION

5.1. INTRODUCTION

An implementation plan is a detailed action document that translates a strategic plan into specific, achievable steps.

Its main purpose is to serve as a step-by-step guide for the execution phase, ensuring your team has clear answers to the who, what, when, how, and why of the project before beginning work. In essence, it is the bridge that turns strategy into actionable tasks.

5.2. CODING

Coding, in simpler terms, is the language used by computers to understand our commands and, therefore, process our requests. It means using the programming language to get the computer to behave as desired. Each line of the code is a set of instructions for the computer. A set of codes form a script, and a set or dozens of sets, form a program.

5.2.1. CODING STANDARDS

A coding standard gives a uniform appearance to the codes written by different engineers. It improves readability, and maintainability of the code and it reduces complexity also. It helps in code reuse and helps to detect error easily.

5.3. CODE VALIDATION AND OPTIMIZATION

5.3.1. DEBUGGING

Debugging, in computer programming and engineering, is a multistep process that involves identifying a problem, isolating the source of the problem, and then either correcting the problem or determining a way to work around it. The final step of debugging is to test the correction or workaround and make sure it works.

6. SYSTEM TESTING

6.1. INTRODUCTION

The purpose of the system testing is to identify and correct errors in the candidate system. Testing is an important element of the software quality assurance and represents the ultimate review of specification, design and coding. The increasing visibility of the software as a system element and the costs associated with a software failure are motivated forces for well-planned through testing. Software testing is a critical element of software quality assurance and represents the ultimate quality review of specifications, design and code generation. Once the source code has been generated, the program should be executed before the customer gets it with the specific intend of finding and removing all errors, test must be designed using disciplined techniques. Testing technique provides the systematic guidance for designing tests. To uncover the errors in the program behavior function and performance the following steps to be done:

- Execute the integral logic of the software components.
- Execute the input and output domains of the program to uncover errors.
- During testing the system is used experimentally to ensure that the software does not fail, i.e., it will run according to the specification and in the way the user exports.
- Preparation of test data plays n vital rule in the system testing. Different set of test data are generated and the system under study is tested using that data.
- While testing using test data errors are again uncovered and corrected using different testing techniques.

System testing was conducted in order to detect errors and for comparing then the final system with the requirement specification report. That is, whether the system meets requirements.

During testing the software was executed with a set of test cases and the output of the program for the test cases was evaluated to determine if the program is performing as it was expected to Testing presents, an interesting challenge for the software engineer attends to hold software from an abstract concept to an acceptable implementation. In testing engineer creates a series of test cases that occurs when errors are uncovered. Testing is the process of executing a program for finding errors. A good test is one that has high probability of finding an uncovered error. The turn error is used to refer the difference between the actual output of the software to fail to perform its required function. Software reliability is defined as the required function.

Software reliability is defined as the probability that the software will not undergo failure for a specified time under specified condition. Failure is the inability of a system or a component to perform a required function according to its specification. Different levels of testing were employed for software to make an error free, fault free and reliable. Basically, in software testing three type of testing methods are adopted.

- Unit testing
- Integration testing
- System testing

6.2. TYPES OF TESTING

6.2.1. UNIT TESTING

In unit testing the analysis tests the programs making a system. Unit testing gives stress on the modules independently of one another, to find errors. This helps the tester in detecting errors in coding and logic that are within that module alone. The errors resulting from the interaction between modules are initially avoided. Unit testing can be performed from the bottom up, starting with smallest and lowest-level modules and proceeding one at a time. For each module in bottom-up testing a short program is used to execute the module and provides the needed data, so that the module is asked to perform the way it will when embedded within the larger system. Unit testing deals with testing a unit testing a unit as a whole. This would test the interaction of many functions but confine the test within one unit. The exact scope of a unit is left to interpretation. Supporting test code, sometimes called scaffolding, may be necessary to support an individual test. This type of testing is driven by the architecture and implementation teams. This focus is also called back-box testing because only the details of the interface are visible to the test. Limits that are global to a unit are tested here.

In software testing, one particular test may need some supporting software. This software established an environment around the test. Only when this environment is established can a correct evolution of the test take place. The scaffolding software may establish state and values for data structures as well as providing dummy external functions for the test. Different scaffolding software rarely is considered part of the system. Sometimes the scaffolding software becomes larger than the system software being tested. Usually, the scaffolding software is not of the same quality as the system

software and frequently is quite fragile.

A small change in the change may much larger changes in the scaffolding. This testing level can be simply defined as integrating and then testing i.e., here, many unit tested modules are combined into subsystems, which are then tested. Integration testing aims at whether the modules can be integrated properly. Hence, the emphasis is on testing interfaces between modules. This testing activity can be considered testing the design.

6.2.2. SYSTEM TESTING

The important and essential part of the system development phase, after designing and developing the software is system testing. Theoretically, a newly designed system should have all the parts or sub systems are in working order, but in reality, each sub-system into one pool and test the whole system to determine whether it meets the user requirements. This is the last change to detect and correct errors before the system is installed for user acceptance testing. The purpose testing is to consider all the likely variations to which it will be subjected and then push the system to its limits.

6.2.3. INTEGRATION TESTING

This testing level can be simply defined as integrating and then testing i.e.,here, many unit tested modules are combined into subsystems, which are then tested. Integration testing aims at whether the modules can be integrated properly. Hence, the emphasis is on testing interfaces between modules. This testing activity can be considered testing the design.

7. SYSTEM MAINTENANCE

7.1. INTRODUCTION

System maintenance is an umbrella term that encompasses various forms of computer maintenance needed to keep a system running. The two main components of system maintenance are preventive and corrective maintenance. Preventive maintenance involves taking measures to help keep the system functioning, whereas corrective maintenance involves the replacement or repair of a system or its components after they have already failed.

7.2. MAINTENANCE

Maintenance projects can be used for enhancement or defect fixing. Enhancement projects are related to implementing new developments. It is basically modifications of software after it has been delivered in order to correct errors, to improve performance or other properties of IT systems. The key problems of software maintenance in IT systems include both management and technical problems. Important management issues are: adjusting to customer priorities, appropriate system maintenance personnel, cost estimation. The key technical issues are limited understanding, impact analysis, testing, measuring system maintainability, and infrastructure maintenance. It is well known that the guarantee of business survival is its continuous development and improvement of solutions, and thus also the continuous and adequate development of IT systems, combined with maintenance work carried out regularly on all components of the production environment.

8. FUTURE ENHANCEMENT AND SCOPE FOR FURTHER DEVELOPMENT

8.1. INTRODUCTION

The TiCare system is designed to replace inefficient paper-based booklets for pet health. It provides a secure online database for storing all pet and owner information, which can be easily managed by an administrator. The primary goal of this project is to give pet owners a reliable platform to ensure their pets never miss a crucial vaccination.

8.2. MERITS OF THE SYSTEM

- Maintenance is easy and performance is good.
- It is user-friendly and easy to understand.
- It reduces the time and effort required to manage records.
- Provides a centralized database for all pet and vaccination data.
- Ensures higher security and reduces the risk of data loss.
- Automates reminders to ensure timely vaccinations.
- Data is accessible 24/7 from any location.
- Minimizes human error and improves the accuracy of health records.

8.3. LIMITATIONS OF THE SYSTEM

- There is no separate portal for veterinarians or clinics to log in and officially verify the records.
- It is a web-based application only and does not have a native mobile app for features like push notifications.
- The scope is currently limited to vaccinations and does not cover a pet's complete health history (e.g., illnesses, medications, allergies).
- The system does not have an offline mode; data cannot be accessed or updated without an internet connection.

8.4. FUTURE ENHANCEMENT OF THE SYSTEM

The TiCare Pet Vaccination Management System successfully achieves its primary goal of providing a reliable and user-friendly platform for pet owners to manage their pets' vaccination schedules. The current system serves as a strong foundation, replacing inefficient manual record-keeping with a centralized and secure web application.

While the current version is a complete and functional product, there is significant scope for further development. The following enhancements could be implemented in the future to expand its capabilities and provide even greater value to users.

Future Enhancements

- **Intelligent Vaccine Suggestions:** Implement an engine to suggest vaccination schedules based on the pet's age, breed, and history.
- **Veterinary Clinic Locator:** Integrate a mapping service to help users find and get directions to nearby veterinary clinics.
- **Dedicated Veterinarian Portal:** Create a separate, secure portal for veterinarians to directly manage and update their patients' records.
- **Online Appointment Booking:** Develop a full appointment booking system for users to schedule visits with clinics directly through the application.
- **Native Mobile App (iOS/Android):** Build a dedicated mobile application to provide push notifications and a better user experience.
- **Comprehensive Digital Health Record:** Expand the system to store a pet's complete health history, including allergies, medications, and past illnesses.

9. CONCLUSION

9.1. CONCLUSION

This project was undertaken to solve the challenges of traditional, paper-based pet record-keeping, which is often inefficient and prone to error. The developed system, "TiCare," successfully addresses these issues by delivering a centralized, secure, and user-friendly web platform. Key achievements include the creation of a robust database for pet and owner information, the implementation of an automated reminder system, and the development of a comprehensive administrative module. While acknowledging the current limitations, the project serves as an excellent foundation with significant scope for future enhancements like a dedicated veterinarian portal and a native mobile app. In conclusion, the TiCare system effectively meets its goal of creating a reliable and efficient tool for modern pet healthcare, demonstrating the value of digital solutions in promoting animal well-being.

10. BIBLIOGRAPHY

10.1. Books:

- Pressman, R. S., & Maxim, B. R. (2019). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- Welling, L., & Thomson, L. (2017). *PHP and MySQL Web Development* (5th ed.). Addison-Wesley Professional.
- Fowler, M. (2003). *UML Distilled: A Brief Guide to the Standard Object Modeling Language* (3rd ed.). Addison-Wesley Professional.
- Beaulieu, A. (2020). *Learning SQL* (3rd ed.). O'Reilly Media.

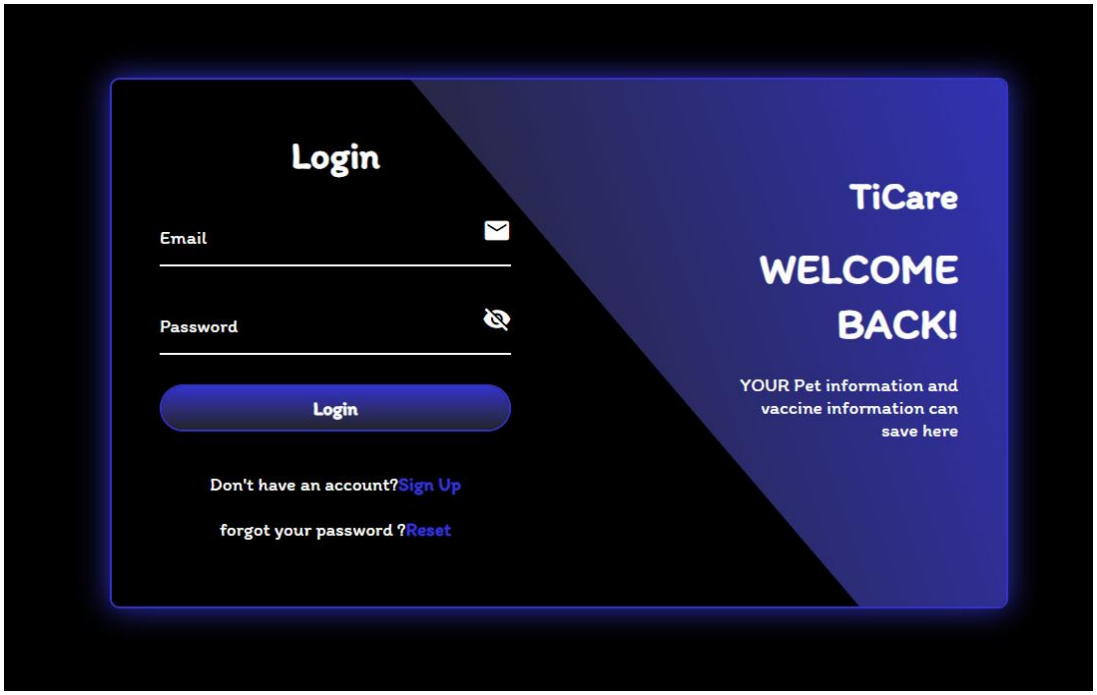
10.2. Web Resources:

- The PHP Group. (2025). *PHP Manual*. Retrieved from https://www.php.net/manual/en/
- Oracle Corporation. (2025). *MySQL Documentation*. Retrieved from https://dev.mysql.com/doc/
- Mozilla Developer Network (MDN). (2025). *Web Docs: HTML, CSS, JavaScript*. Retrieved from https://developer.mozilla.org/
- W3Schools. (2025). *PHP and SQL Tutorials*. Retrieved from https://www.w3schools.com/php/

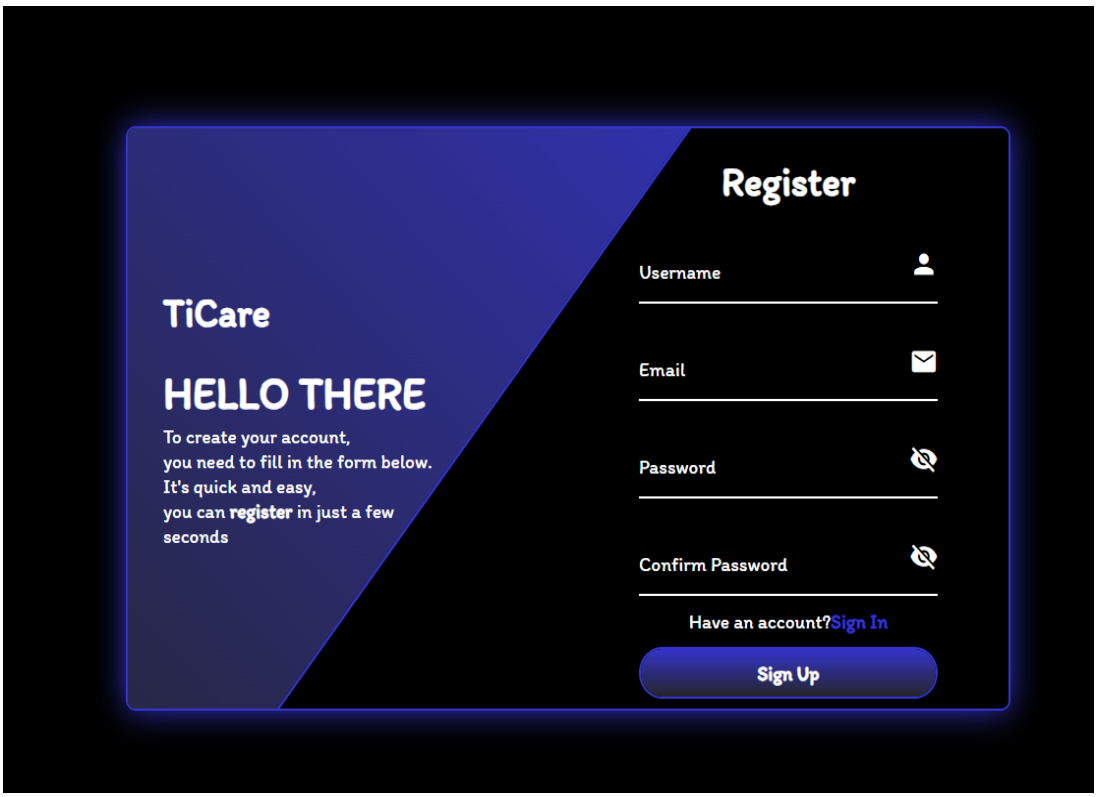
11. ANNEXURES

11.1. SCREENSHOTS

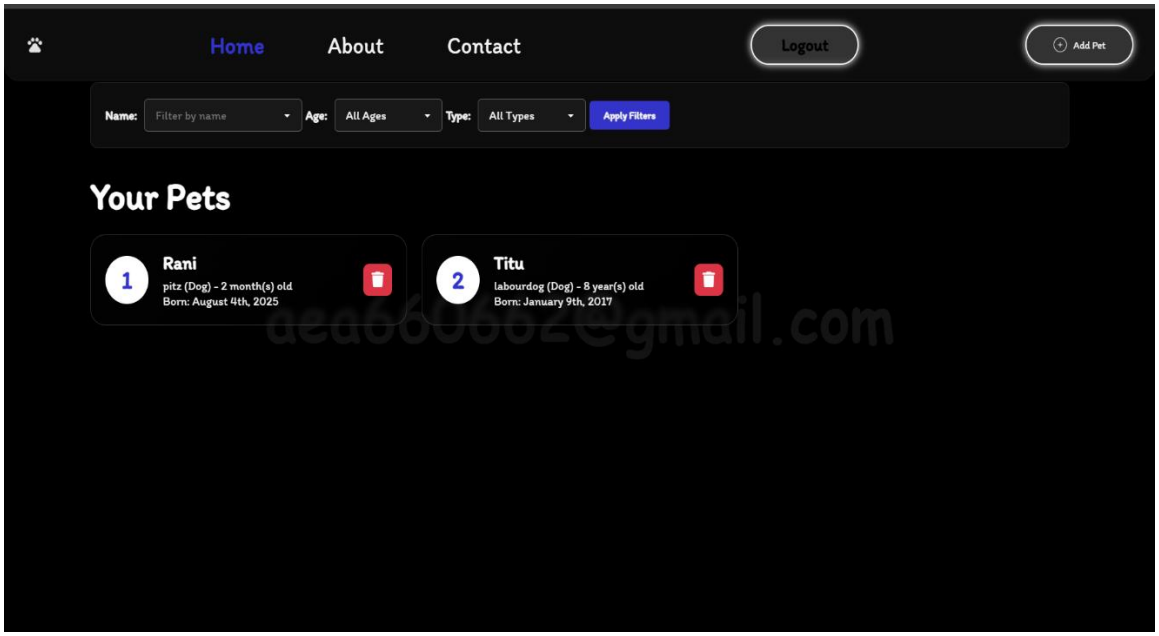
LOGIN PAGE



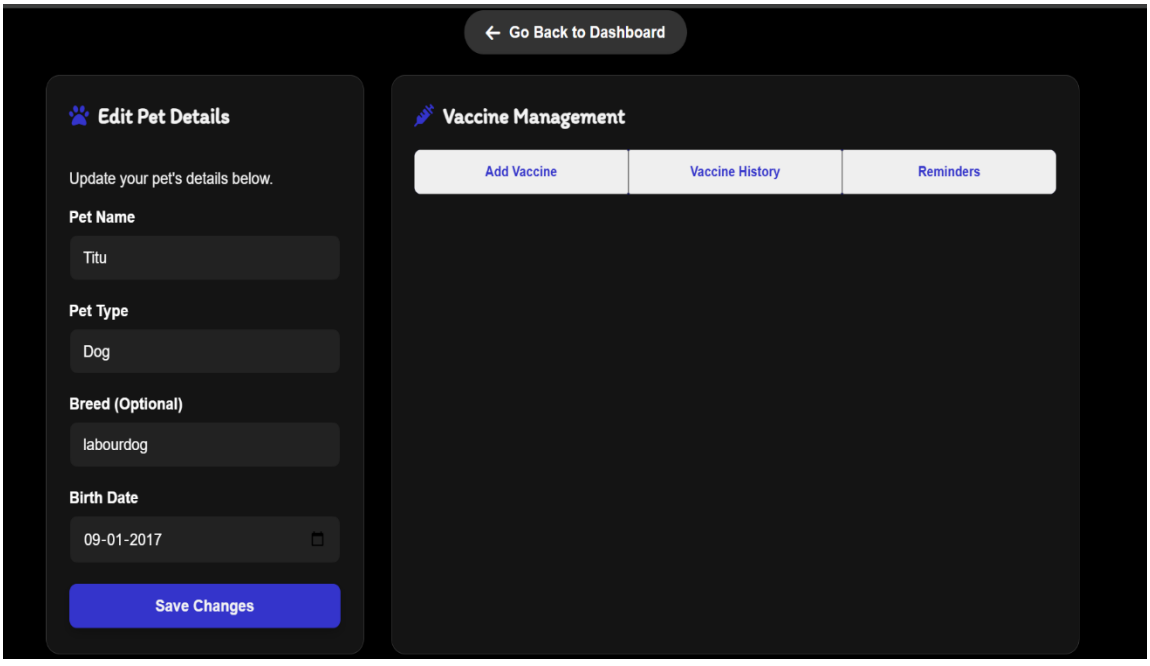
REGISTER PAGE



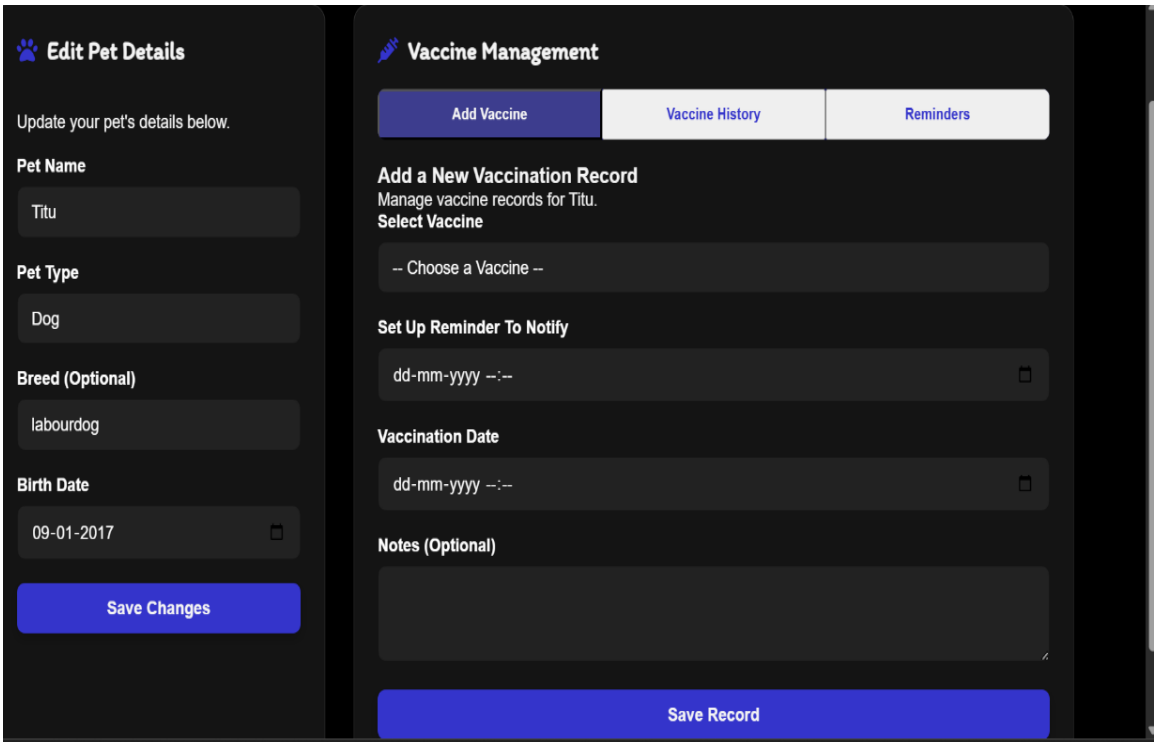
HOME PAGE



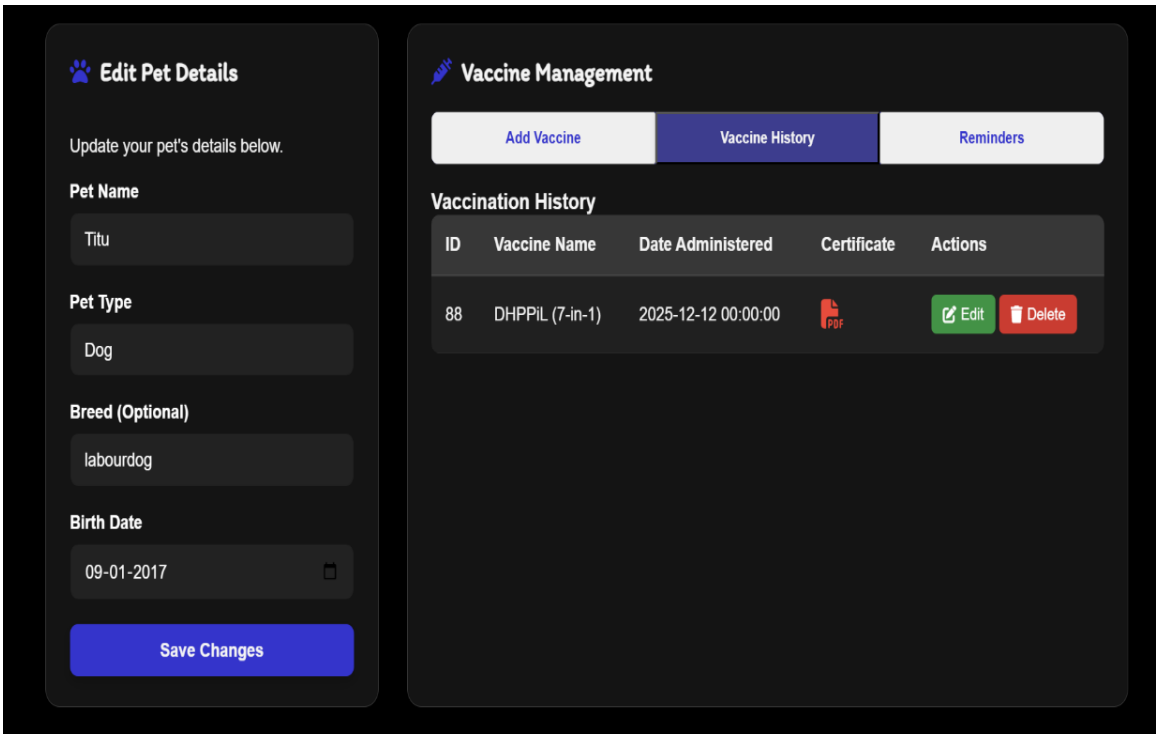
PET DETAILS & VACCINATION MANAGEMENT



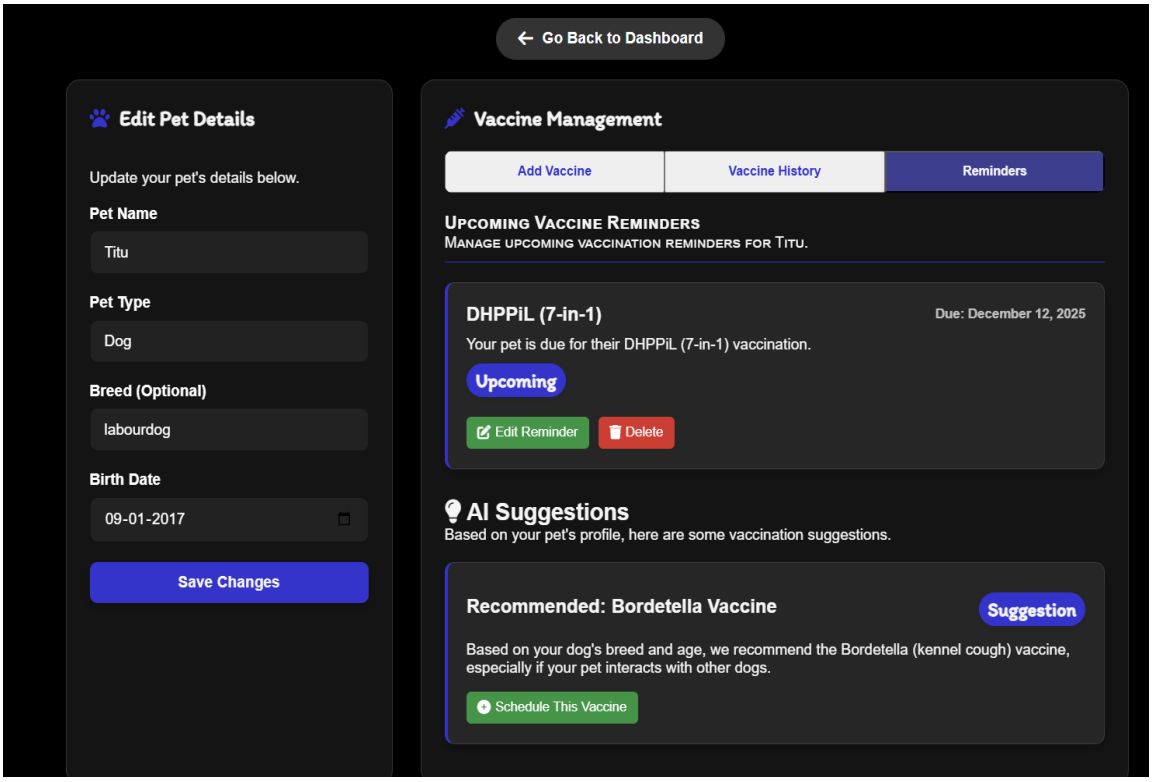
ADD VACCINE



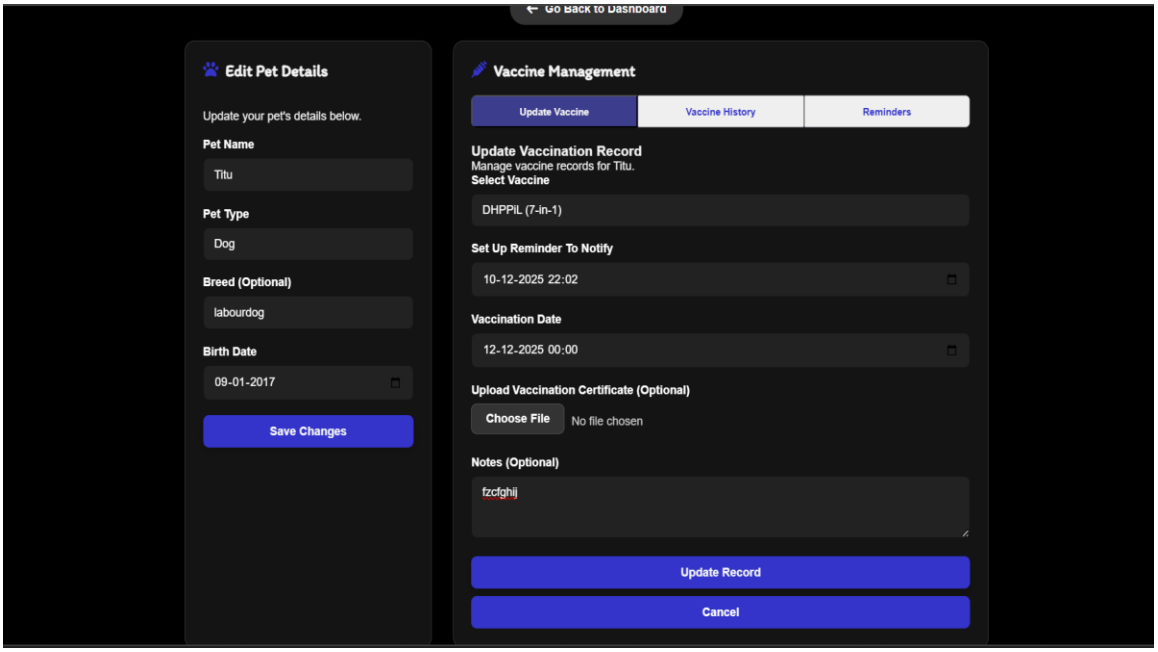
VACCINE TABLE



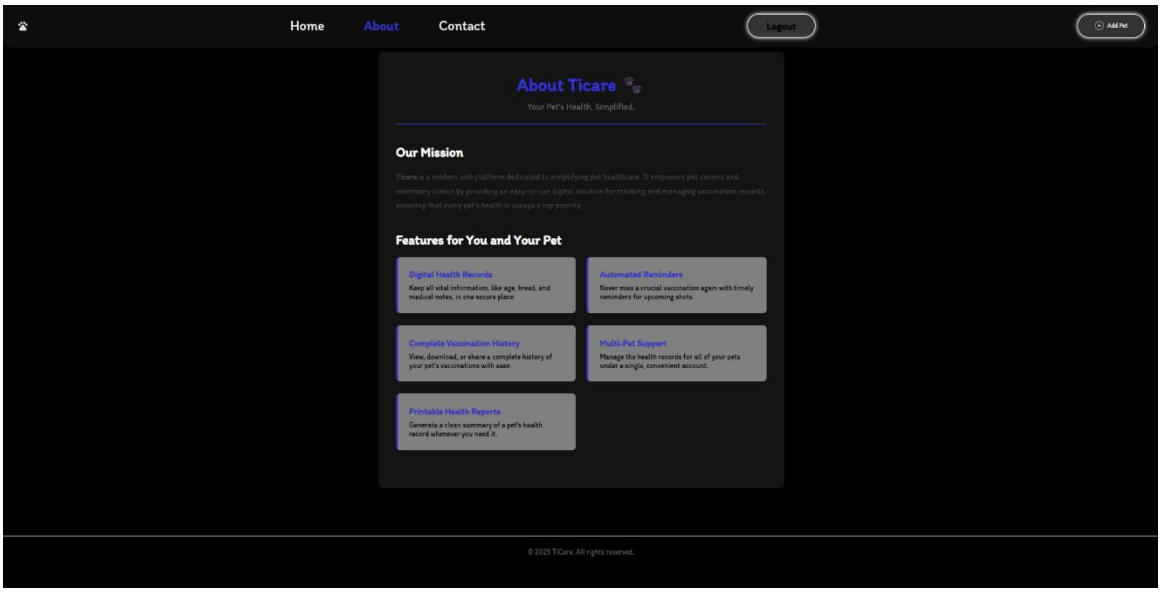
UP COMING VACCINE REMINDER



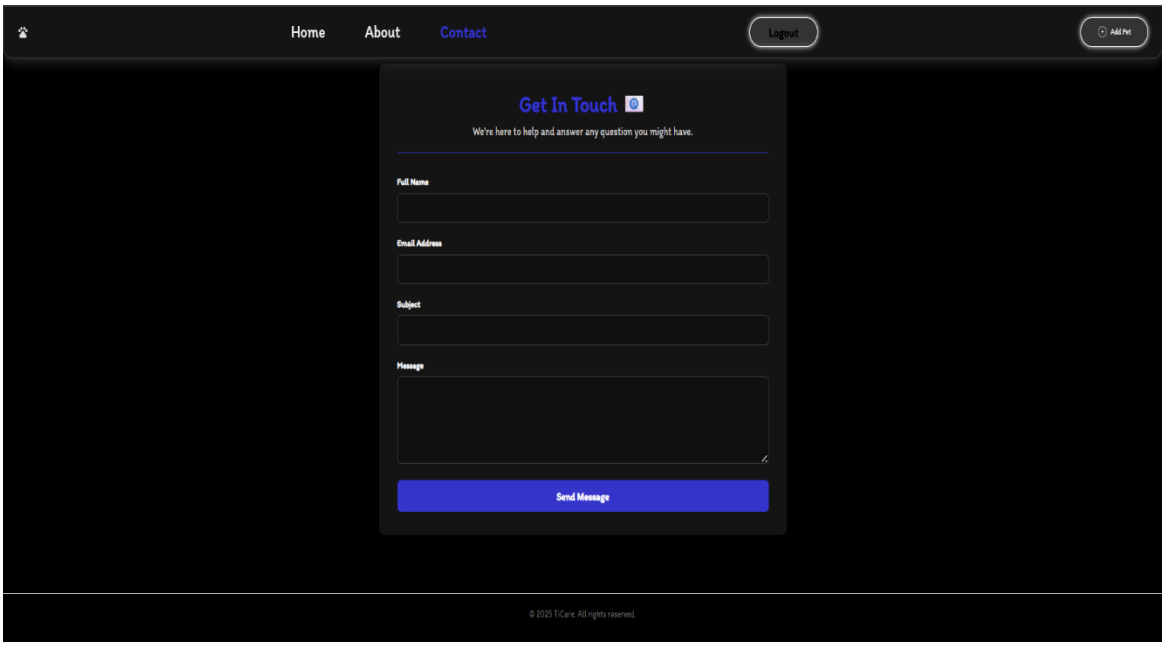
UPDATE VACCINE RECORD



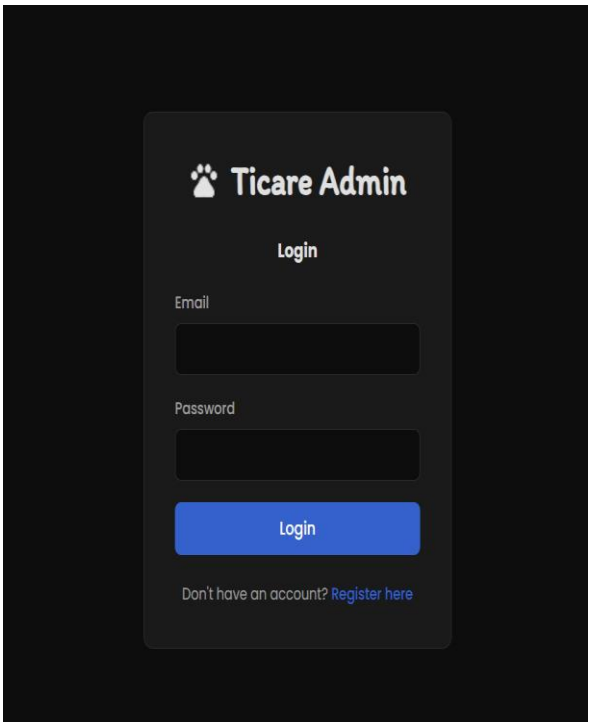
ABOUT THE WEB APPLICATION

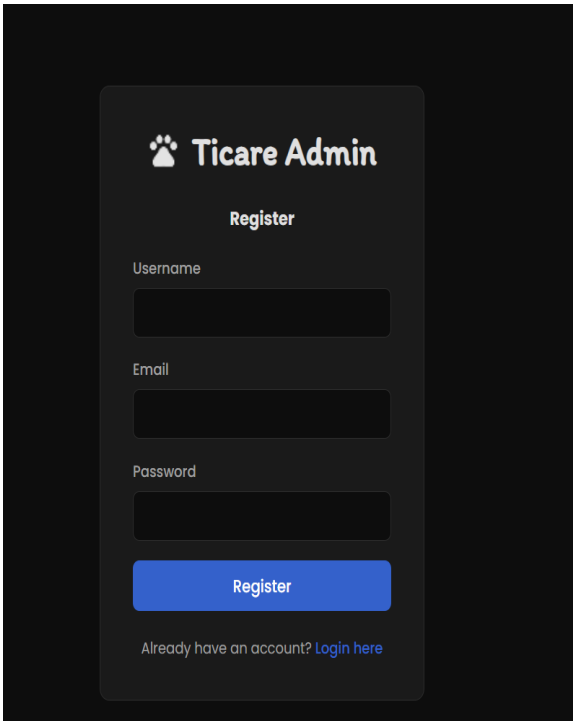


CONTACT/HELP LINE

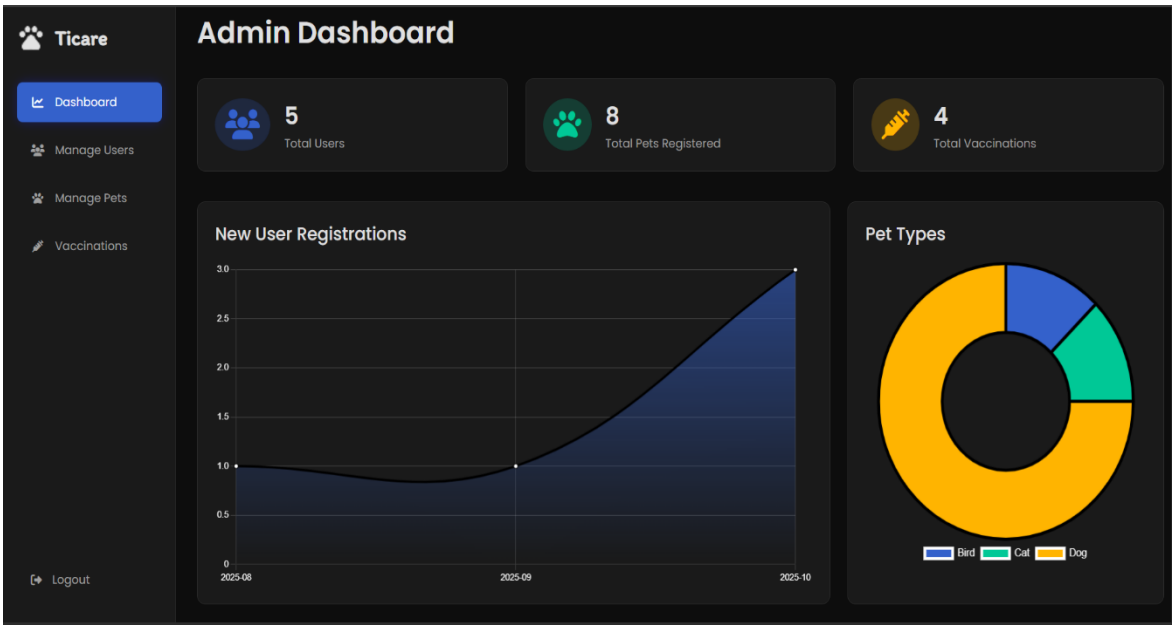


ADMIN LOGIN / REGISTER

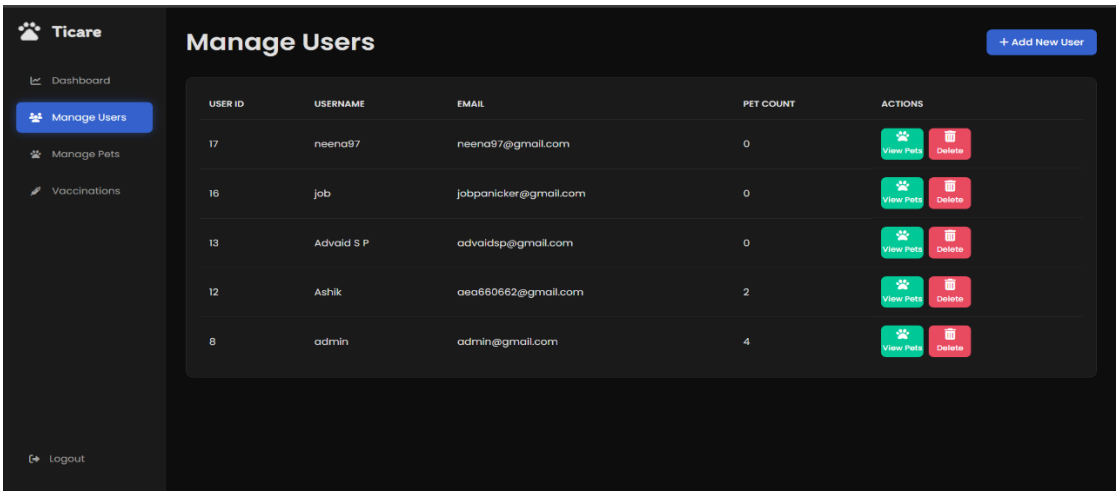


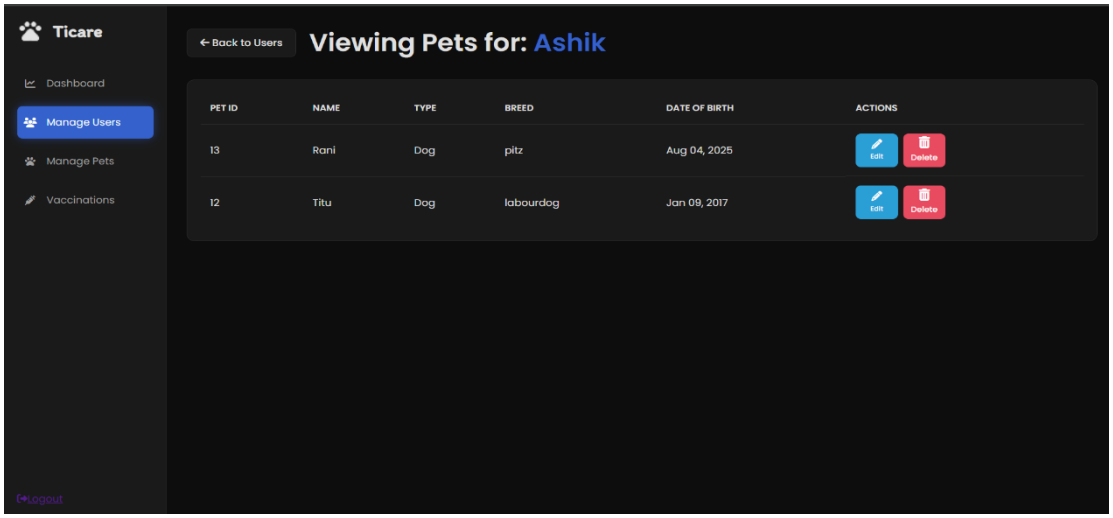


DASHBORD

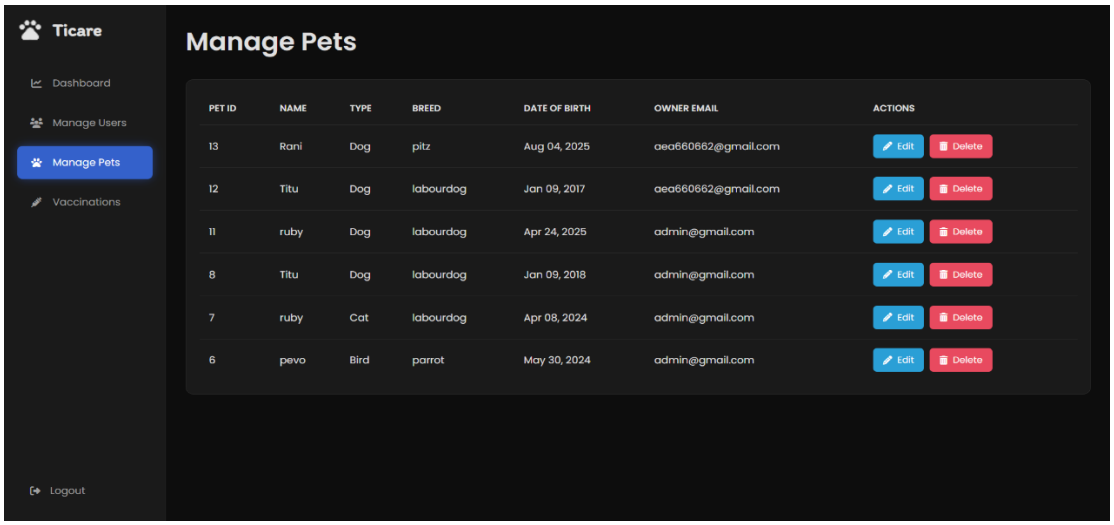


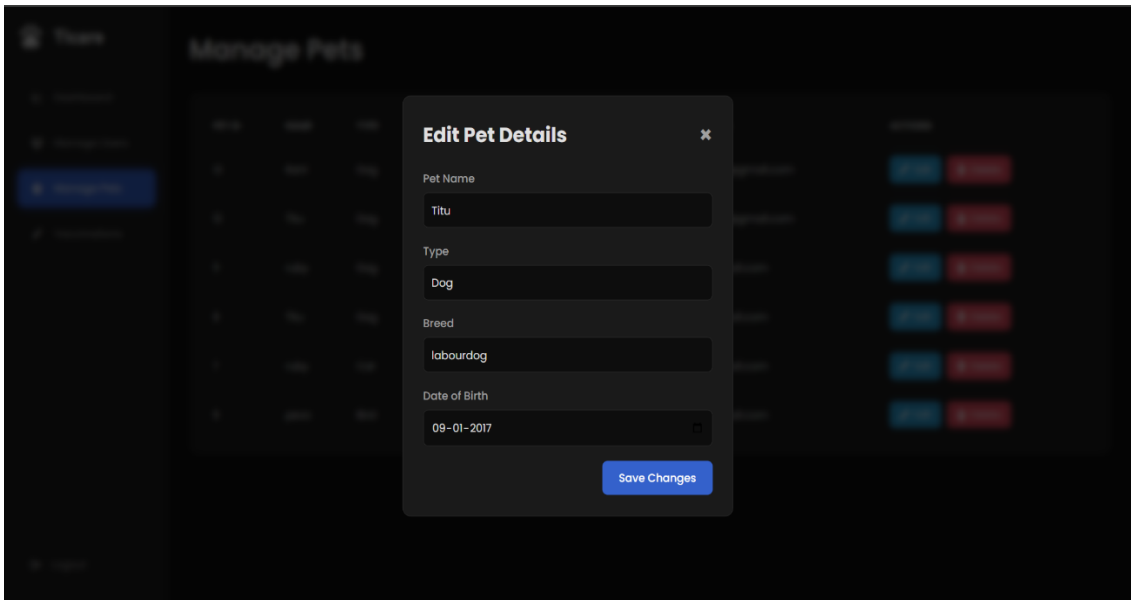
MANAGE USERS



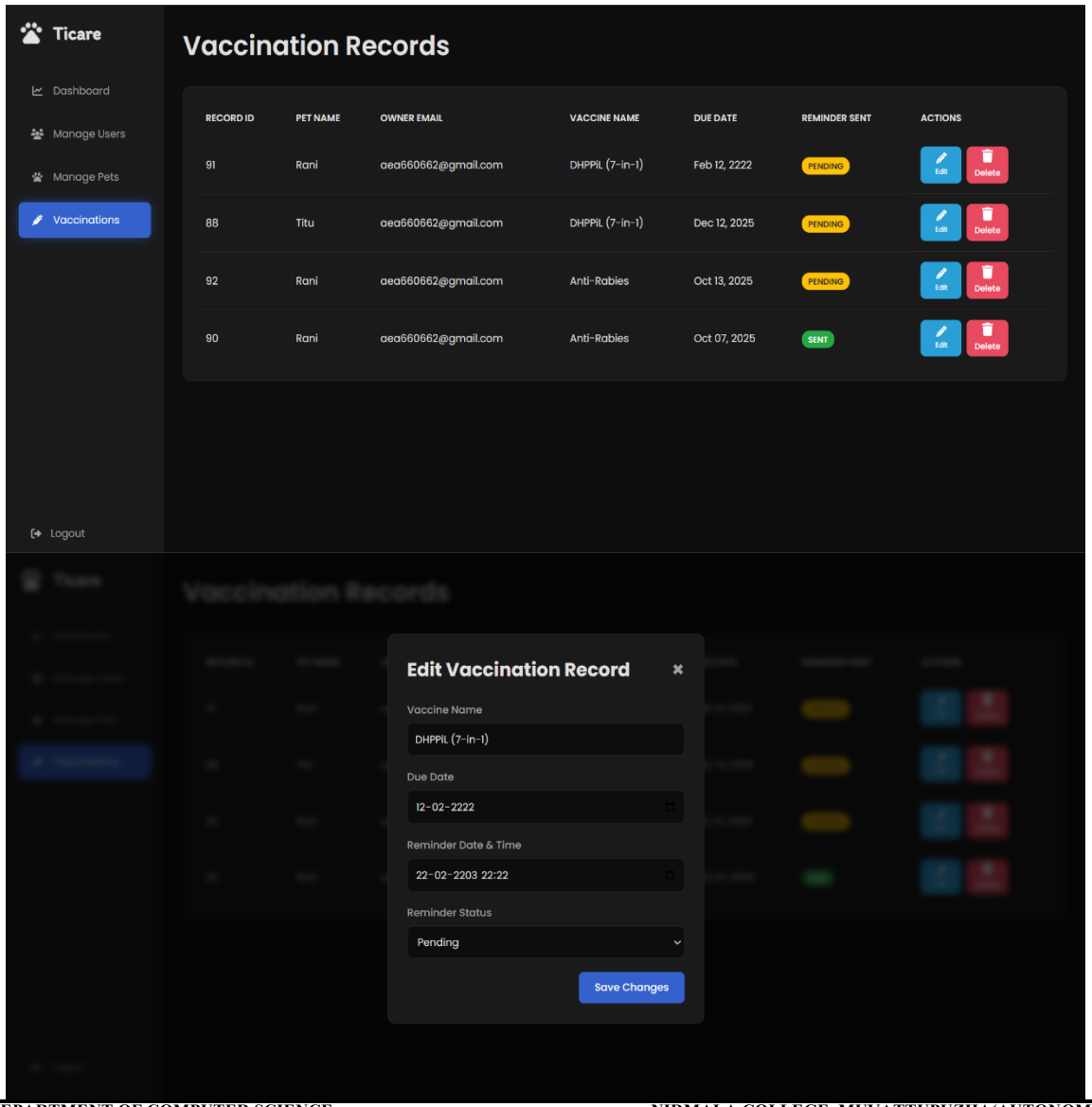


MANAGE PETS





VACCINATIONS



11.2. CODING

USER LOGIN & REGISTRATION

```
<?php
session_start();
error_reporting(E_ALL);
ini_set('display_errors', 1);
ini_set('html_errors', 1);
include "connection/connect.php";
require "vendor/autoload.php";
use Firebase\JWT\JWT;
use Firebase\JWT\Key;
setcookie('jwt_token', '', time() - 3600, '/');
$secret_key = "your_secret_key";
$issuer_claim = "localhost";
$audience_claim = "localhost";
$issuedat_claim = time();
$expire_claim = $issuedat_claim + 3600;

if (isset($_POST['register'])) {
    // All your registration PHP logic here...
    $username = $_POST['username'];
    $email = $_POST['email-register'];
    $raw_password = $_POST['password-register'];
    $confirm_password = $_POST['confirm-password-register'] ?? '';
    $errors = [];

    // ... (rest of the registration validation and insertion code)
}

if (isset($_POST['login'])) {
    // All your login PHP logic here...
    $email = $_POST['email-login'];
    $password_input = $_POST['password-login'];

    // ... (rest of the login validation and JWT creation code)
}

// Include the header file
include 'header.php';
?>

<div class="container">
    <div class="curver-shape"></div>
    <div class="curver-shape2"></div>
    <div class="form-box Login">
        <h2 class="animation" style="--D:0">Login</h2>
        <form action="index.php" method="post" id="loginForm">
            <div class="input-box animation" style="--D:1">
```

```
            <input type="email" name="email-login" id="email-login" required>
              <label for="email-login">Email</label>
                 <i class="material-icons icon-email">mail</i> <span class="error-message"
id="emailLoginError"></span>
            </div>
            <div class="input-box animation" style="--D:2">
                <input type="password" name="password-login" id="password-login"
required>
                    <label for="password-login">Password</label> <i class="material-icons
password-toggle" data-target="password-login">visibility_off</i>
                       <span class="error-message" id="passwordLoginError"></span> </div>
            <div class="input-box animation" style="--D:3">
              <button class="btn" type="submit" name="login">Login</button>
            </div>
            <div class="regi-link">
              <p class="animation" style="--D:4">Don't have an account?<a href="#"
class="signUp">Sign Up</a></p><br>
              <p class="animation" style="--D:4">forgot your password ?<a
href="pages/forgot.php" class="signUp">Reset</a></p>
            </div>
          </form>
        </div>
        <div class="info-content Login">
           <h1 class="animation" style="--D:1.5">TiCare</h1>
          <br>
          <h2 class="animation" style="--D:1">WELCOME BACK!</h2>
          <br>
          <p class="animation" style="--D:2">YOUR Pet information and vaccine information
can save here </p>
        </div>
        <div class="form-box Register">
          <h2 class="animation" style="--li:1">Register</h2>
          <form action="index.php" method="post" id="registrationForm">
            <div class="input-box animation" style="--li:2">
              <input type="text" name="username" id="username-register" required>
              <label for="username-register">Username</label>
              <i class="material-icons">person</i>
              <span class="error-message" id="usernameRegisterError"></span>
            </div>
            <div class="input-box animation" style="--li:3">
              <input type="email" name="email-register" id="email-register" required>
              <label for="email-register">Email</label>
              <i class="material-icons icon-email">mail</i> <span class="error-message"
id="emailRegisterError"></span>
            </div>
            <div class="input-box animation" style="--li:4">
              <input type="password" name="password-register" id="password-register"
required>
              <label for="password-register">Password</label>
              <i class="material-icons password-toggle" data-target="password-
register">visibility_off</i>
```

```
            <span class="error-message" id="passwordRegisterError"></span>
          </div>
          <div class="input-box animation" style="--li:4.5" id="confirm-password-register-box">
            <input type="password" name="confirm-password-register" id="confirm-password-register" required>
            <label for="confirm-password-register">Confirm Password</label>
            <i class="material-icons password-toggle" data-target="confirm-password-register">visibility_off</i>
             <span class="error-message" id="confirmPasswordRegisterError"></span>
          </div>
          <div class="regi-link animation" style="--li:5">
            <p>Have an account?<a href="#" class="signIn">Sign In</a></p>
          </div>
          <div class="input-box animation register-button-box" style="--li:6">
            <button class="btn" type="submit" name="register">Sign Up</button>
          </div>
        </form>
      </div>
      <div class="info-content Register">
        <h1 class="animation" style="--info:0">TiCare</h1>
        <br>
        <h2 class="animation" style="--info:1">HELLO THERE</h2>
        <p class="animation" style="--info:2">To create your account, <br>
          you need to fill in the form below. <br>
          It's quick and easy, <br>
          you can <strong>register</strong> in just a few <br>seconds </p>
      </div>
    </div>

<?php
// Include the footer file
include 'footer.php';
?>
```

SEND REMINDER

```
      <?php
    require __DIR__ . "/../vendor/autoload.php";
    use PHPMailer\PHPMailer\PHPMailer;
    use PHPMailer\PHPMailer\Exception;

    include __DIR__ . "/../connection/connect.php";
    if ($con->connect_error) {
        die("Database connection failed: " . $con->connect_error);
    }

    echo "Cron Job Started at: " . date('Y-m-d H:i:s') . "\n";

    try {
        $sql = "SELECT * FROM pet_vaccination_records WHERE reminder_datetime <= NOW() AND reminder_sent = 0";
        $stmt = $con->prepare($sql);
```

```
$stmt->execute();
$reminders = $stmt->get_result()->fetch_all(MYSQLI_ASSOC);

if (count($reminders) === 0) {
    echo "No reminders to send.\n";
    exit;
}

echo "Found " . count($reminders) . " reminder(s) to send.\n";

foreach ($reminders as $reminder) {
    $record_id = $reminder['id'];
    $owner_email = $reminder['owner_email'];
    $pet_name = $reminder['pet_name'];
    $vaccine_name = $reminder['vaccine_name'];
    $due_date = date("F j, Y, g:i a", strtotime($reminder['reminder_datetime']));

    $mail = new PHPMailer(true);
    try {
        $mail->isSMTP();
        $mail->Host       = 'smtp.gmail.com';
        $mail->SMTPAuth   = true;
        $mail->Username   = 'ashikaswin1@gmail.com';
        $mail->Password   = '*';
        $mail->SMTPSecure = PHPMailer::ENCRYPTION_STARTTLS;
        $mail->Port       = 587;

        $mail->setFrom('no-reply@yourdomain.com', 'TiCare Pet Reminders');
        $mail->addAddress($owner_email);

        $mail->isHTML(true);
        $mail->Subject = 'Vaccine Reminder for ' . $pet_name;
        $mail->Body    = "
            <html>
            <body>
                <h2>Hi there,</h2>
                <p>This is a friendly reminder that <strong>{$pet_name}</strong> has a
vaccine appointment due for the <strong>{$vaccine_name}</strong>.</p>
                <p><strong>Scheduled Time:</strong> {$due_date}</p>
                <p>Thank you for using TiCare!</p>
            </body>
            </html>";

        $mail->send();
        echo "Email sent successfully to {$owner_email} for pet {$pet_name}.\n";

        $update_sql = "UPDATE pet_vaccination_records SET reminder_sent = 1
WHERE id = ?";
        $update_stmt = $con->prepare($update_sql);
        $update_stmt->bind_param("i", $record_id);
        $update_stmt->execute();
```

```
            echo "Record ID {$record_id} marked as sent.\n";

        } catch (Exception $e) {
            echo "Message could not be sent. Mailer Error: {$mail->ErrorInfo}\n";
        }
    }

} catch (Exception $e) {
    echo "Database error: " . $e->getMessage() . "\n";
}

echo "Cron Job Finished at: " . date('Y-m-d H:i:s') . "\n";
$con->close();
?>
```

ADMIN LOGIN & REGISTER

```
<?php
// Start the session at the very top.
session_start();

// --- 1. DATABASE CONNECTION ---
$servername = "localhost";
$username = "root";
$password = "";
$dbname = "ticareproject";

$conn = new mysqli($servername, $username, $password, $dbname);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Initialize messages and a variable to track which form to show
$register_message = '';
$login_message = '';
$default_form = 'login'; // By default, show the login form

// --- 2. HANDLE FORM SUBMISSIONS ---
if ($_SERVER['REQUEST_METHOD'] === 'POST') {

    // --- HANDLE REGISTRATION ---
    if (isset($_POST['action']) && $_POST['action'] === 'register') {
        $default_form = 'register'; // If registration is attempted, show register form on reload
        $username = $_POST['username'];
        $email = $_POST['email'];
        $password_plain = $_POST['password'];

        if (empty($username) || empty($email) || empty($password_plain)) {
            $register_message = '<div class="message error">All fields are required.</div>';
        } elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            $register_message = '<div class="message error">Invalid email format.</div>';
        } else {
```

```
            $check_email_stmt = $conn->prepare("SELECT id FROM admin WHERE email =
?");
            $check_email_stmt->bind_param("s", $email);
            $check_email_stmt->execute();
            $check_email_stmt->store_result();

            if ($check_email_stmt->num_rows > 0) {
                $register_message = '<div class="message error">An account with this email
already exists.</div>';
            } else {
                $password_hashed = password_hash($password_plain,
PASSWORD_DEFAULT);
                $insert_stmt = $conn->prepare("INSERT INTO admin (username, email,
password) VALUES (?, ?, ?)");
                $insert_stmt->bind_param("sss", $username, $email, $password_hashed);

                if ($insert_stmt->execute()) {
                    $register_message = '<div class="message success">Registration successful!
You can now log in.</div>';
                    $default_form = 'login'; // Switch back to login on success
                } else {
                    $register_message = '<div class="message error">Error: Could not
register.</div>';
                }
                $insert_stmt->close();
            }
            $check_email_stmt->close();
        }
    }

    // --- HANDLE LOGIN ---
    if (isset($_POST['action']) && $_POST['action'] === 'login') {
        $email = $_POST['email'];
        $password = $_POST['password'];

        if (empty($email) || empty($password)) {
            $login_message = '<div class="message error">Email and password are
required.</div>';
        } else {
            $login_stmt = $conn->prepare("SELECT id, username, password FROM admin
WHERE email = ?");
            $login_stmt->bind_param("s", $email);
            $login_stmt->execute();
            $result = $login_stmt->get_result();

            if ($result->num_rows === 1) {
                $user = $result->fetch_assoc();
                if (password_verify($password, $user['password'])) {
                    session_regenerate_id(true);
                    $_SESSION['admin_id'] = $user['id'];
                    $_SESSION['admin_username'] = $user['username'];
```

```
                header("Location: dashboard.php");
                exit();
            } else {
                $login_message = '<div class="message error">Incorrect email or
password.</div>';
            }
        } else {
            $login_message = '<div class="message error">Incorrect email or
password.</div>';
        }
        $login_stmt->close();
    }
  }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Admin Access - Ticare</title>
    <link rel="icon" type="image/png" href="../images/petlogo.png">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link
href="https://fonts.googleapis.com/css2?family=Itim&family=Poppins:wght@300;400;500;6
00;700&display=swap" rel="stylesheet">
    <link rel="stylesheet" href="admin-style.css">
</head>
<body data-default-form="<?php echo $default_form; ?>">
    <div class="form-container">
        <div class="form-header">
            <img src="images/petlogo.png" alt="Ticare Logo">
            <h2>Ticare Admin</h2>
        </div>

        <div id="login-container">
            <h3>Login</h3><br>
            <?php echo $login_message; ?>
            <form method="POST" action="admin_login.php">
                <input type="hidden" name="action" value="login">
                <div class="form-group">
                    <label for="login-email">Email</label>
                    <input type="email" id="login-email" name="email" required>
                </div>
                <div class="form-group">
                    <label for="login-password">Password</label>
                    <input type="password" id="login-password" name="password" required>
                </div>
                <button type="submit" class="submit-btn">Login</button>
            </form>
```

```
            <p class="toggle-link">Don't have an account? <a href="#" id="show-
register">Register here</a></p>
        </div>

        <div id="register-container">
            <h3>Register</h3><br>
            <?php echo $register_message; ?>
            <form method="POST" action="admin_login.php">
                <input type="hidden" name="action" value="register">
                <div class="form-group">
                    <label for="register-username">Username</label>
                    <input type="text" id="register-username" name="username" required>
                </div>
                <div class="form-group">
                    <label for="register-email">Email</label>
                    <input type="email" id="register-email" name="email" required>
                </div>
                <div class="form-group">
                    <label for="register-password">Password</label>
                    <input type="password" id="register-password" name="password" required>
                </div>
                <button type="submit" class="submit-btn">Register</button>
            </form>
            <p class="toggle-link">Already have an account? <a href="#" id="show-
login">Login here</a></p>
        </div>
    </div>
    <script src="admin-script.js"></script>
</body>
</html>
<?php
$conn->close();
?>
```